

SQL *Cheat Sheet*

The Complete Reference for Data Work

`FROM → WHERE → GROUP BY → HAVING → WINDOW → SELECT → DISTINCT → ORDER BY → LIMIT`

WRITTEN BY

Sharahbill Mohammed Abdi

Data / ML Engineer

SECTIONS	APPENDICES	CODE EXAMPLES	TESTED ON
25	3	88	**PostgreSQL 17**



IMPRINT

SQL Cheat Sheet

The Complete Reference for Data Work

AUTHOR Sharahbill Mohammed Abdi

PUBLISHER Sharahbill Mohammed Abdi

EDITION First edition, 2026

© 2026 Sharahbill Mohammed Abdi. All rights reserved.

CONTENTS

Inside this reference

PART 01 **Foundations** 5

01 Sample Schema Used in the Examples 6

02 How a Query Is Processed: Logical Order of Execution 8

03 SELECT Basics 9
Select columns, all columns, expressions and aliases · DISTINCT: unique values · Comments and quoting

04 Filtering Rows (WHERE) 11
Comparison operators · AND, OR, NOT and precedence · BETWEEN, IN · Pattern matching with LIKE · NULL handling in filters

05 Sorting and Limiting 13
ORDER BY · LIMIT, OFFSET and FETCH

PART 02 **Aggregation & Logic** 15

06 Aggregation, GROUP BY and HAVING 16
Aggregate functions · Median and percentiles · GROUP BY · HAVING: filter groups · Conditional aggregation and FILTER · ROLLUP, CUBE and GROUPING SETS: subtotals · String aggregation

07 Conditional Logic: CASE, COALESCE, NULLIF 19
CASE expression · COALESCE, NULLIF, GREATEST, LEAST

PART 03 **Combining Data** 21

08 Joins 22
The join types · Joining several tables · USING and NATURAL · Semi-joins and anti-joins (EXISTS / NOT EXISTS) · Join pitfalls

09 Subqueries and CTEs 25
Subquery types · Common Table Expressions (CTEs) · Recursive CTEs: hierarchies and series

10 Window Functions (Critical for Data Science) 28
Ranking functions · Top-N per group · LAG and LEAD (time series) · Moving averages and cumulative totals · Window frame specifications · FIRST_VALUE, LAST_VALUE, NTH_VALUE and named windows

11 Set Operations 31

PART 04 **Defining & Changing Data** 33

12 Data Definition (DDL): CREATE, ALTER, DROP 34
Common data types · CREATE TABLE · Create a table from a query, temporary tables · ALTER TABLE · DROP and TRUNCATE

13 Constraints and Keys 37

14 Data Manipulation (DML): INSERT, UPDATE, DELETE 38
INSERT · UPSERT: insert or update if the row exists · UPDATE · DELETE

15 Indexes 41

16 Views and Materialized Views 42

17 Transactions 43

PART 05 **Functions & Patterns** 46

18 Common Functions 47
String functions · Pattern matching with regular expressions · Date and time: current values and extraction · Date arithmetic and truncation · Date ranges and comparisons · Numeric functions · Type conversion (casting) · NULL handling summary

19 Real-World Data Science Patterns 52
Remove duplicates (keep the latest row per key) · Customer segmentation (spend quartile and recency) · Customer cohort analysis (monthly retention) · RFM analysis (Recency, Frequency, Monetary) · Time-series feature engineering · A/B test analysis · Running totals, share of total, gaps

PART 06 **Performance, Practice & Reference** 56

20 Performance Optimization 57
Query optimization tips · Partitioning large tables · Common performance issues

21 Best Practices 58
Coding standards · Security considerations

22 Database-Specific Extensions 61
PostgreSQL: JSON, arrays, full-text search · MySQL specifics · SQL Server specifics

23 Dialect Differences at a Glance 63

24 Quick Reference 64
Order of execution · Statement families · Common data types · Key functions by category

25 Key Tips for Data Professionals 65

BACK MATTER **Appendices** 66

A Sample Data (PostgreSQL) 67

B Corrections Made to the Original Notes 69

C Exporting This Cheat Sheet (from the original notes) 70

ABOUT THIS REFERENCE

SQL Cheat Sheet

The Complete Reference for Data Work

This combines the two original Notion pages ("SQL Cheatsheet" and "SQL COMMANDS") into one deduplicated, corrected and extended reference. Every topic has a short explanation and a runnable example.

HOW TO USE THE EXAMPLES

- Examples use **standard SQL** and were tested on **PostgreSQL 17** against the sample schema in section 1 (load it, plus the sample data in Appendix A, and every PostgreSQL example runs as written).
- Blocks whose first line is a comment such as `-- MySQL` or `-- SQL Server` show the syntax for that database instead. They were written to the vendor documentation but not executed here.
- Dialect differences are flagged with **Dialects:** notes. A side-by-side table is in section 23.
- Corrections made to the original notes are listed in Appendix B.

READING THIS REFERENCE

POSTGRESQL

Standard SQL, tested on PostgreSQL 17 against the sample schema.

POSTGRESQL

Block whose first line names PostgreSQL syntax.

MYSQL

First line is a `-- MySQL` comment: written to the vendor documentation, not executed here.

SQL SERVER

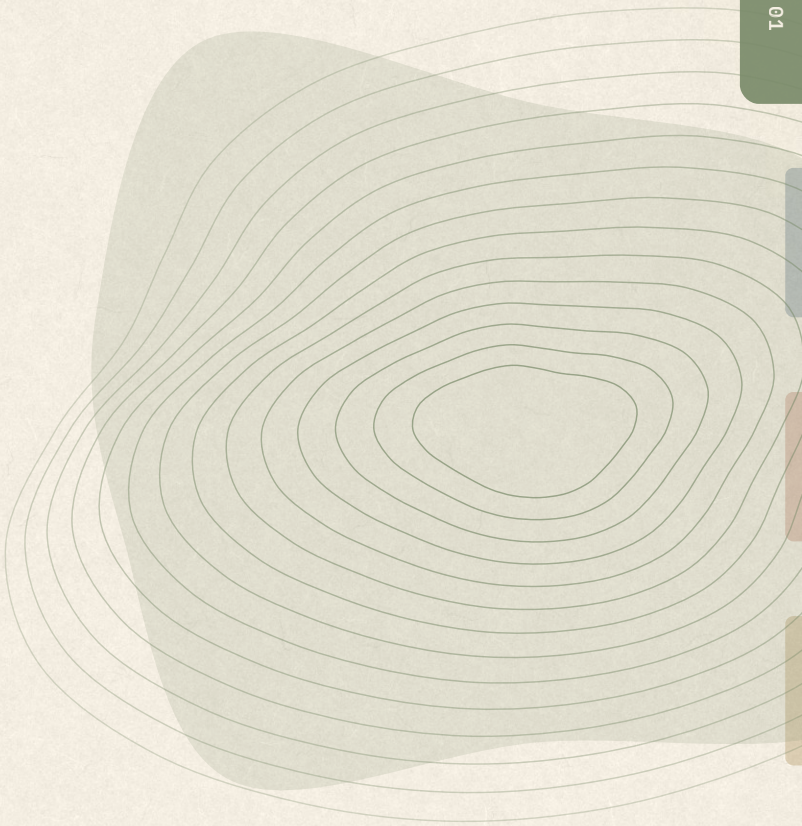
First line is a `-- SQL Server` comment: written to the vendor documentation, not executed here.

DIALECTS | TIP | WATCH OUT | NOTE | REFERENCE

Every code block carries a reference number (`section.block`). Coloured tabs on the right edge mark the six parts and the appendices; the contents, part openers and "In this section" rows are live links.

01

Foundations



01 **Sample Schema Used in the Examples** 6

02 **How a Query Is Processed: Logical Order of Execution** 8

03 **SELECT Basics** 9

Select columns, all columns, expressions and aliases · DISTINCT: unique values · Comments and quoting

04 **Filtering Rows (WHERE)** 11

Comparison operators · AND, OR, NOT and precedence · BETWEEN, IN · Pattern matching with LIKE · NULL handling in filters

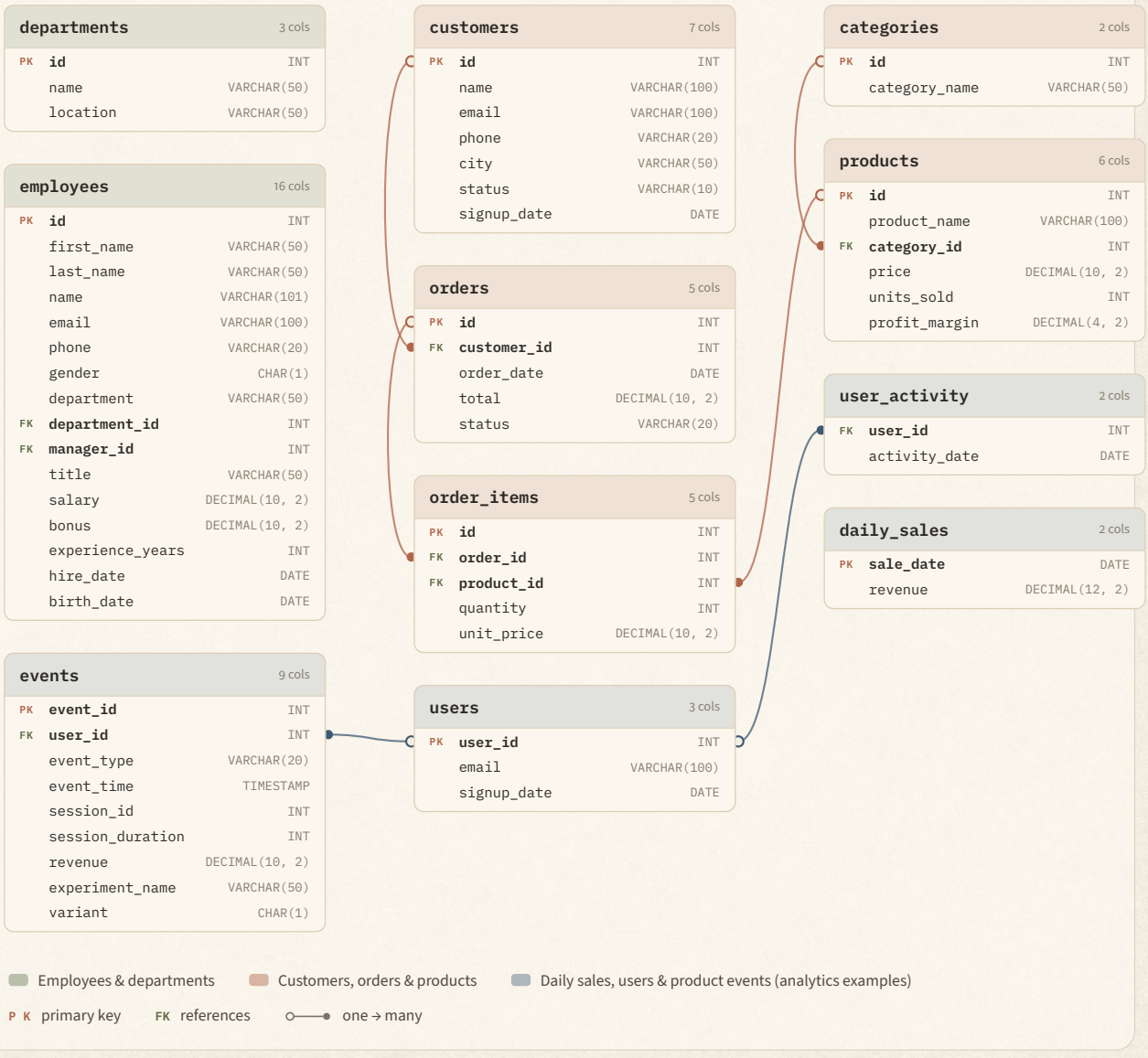
05 **Sorting and Limiting** 13

ORDER BY · LIMIT, OFFSET and FETCH

01 Sample Schema Used in the Examples

A small company database: employees and departments, customers placing orders for products, plus daily sales, users and product events for analytics examples.

FIGURE *Sample schema · 11 tables · foreign-key relationships as declared in the DDL*



POSTGRESQL 01.01

```postgresql
CREATE TABLE departments (
    id         INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name       VARCHAR(50) NOT NULL UNIQUE,
    location   VARCHAR(50)
);
```

```sql
CREATE TABLE employees (
    id              INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name      VARCHAR(50)  NOT NULL,
    last_name       VARCHAR(50)  NOT NULL,
    name            VARCHAR(101) NOT NULL,          -- full name (first + last)
    email           VARCHAR(100) UNIQUE,
    phone           VARCHAR(20),
    gender          CHAR(1),                        -- 'M', 'F' or NULL
    department      VARCHAR(50),                    -- denormalised copy, handy for GROUP BY demos
    department_id   INT REFERENCES departments(id),
    manager_id      INT REFERENCES employees(id),
    title           VARCHAR(50),
    salary          DECIMAL(10, 2) CHECK (salary >= 0),
    bonus           DECIMAL(10, 2),
    experience_years INT,
    hire_date       DATE,
    birth_date      DATE
);

CREATE TABLE customers (
    id          INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(100),
    phone       VARCHAR(20),
    city        VARCHAR(50),
    status      VARCHAR(10) DEFAULT 'active',      -- 'active' / 'inactive'
    signup_date DATE
);

CREATE TABLE categories (
    id            INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL
);

CREATE TABLE products (
    id            INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    product_name  VARCHAR(100) NOT NULL,
    category_id   INT REFERENCES categories(id),
    price         DECIMAL(10, 2),
    units_sold    INT,
    profit_margin DECIMAL(4, 2)                    -- 0.35 = 35 %
);

CREATE TABLE orders (
    id          INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    customer_id INT REFERENCES customers(id),
    order_date  DATE NOT NULL,
    total       DECIMAL(10, 2),
    status      VARCHAR(20)                        -- 'paid', 'shipped', 'cancelled'
);

CREATE TABLE order_items (
    id         INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    order_id   INT REFERENCES orders(id),
    product_id INT REFERENCES products(id),
    quantity   INT,
    unit_price DECIMAL(10, 2)
);
CREATE TABLE daily_sales (
    sale_date  DATE PRIMARY KEY,
```

```
    revenue    DECIMAL(12, 2)
);

CREATE TABLE users (
    user_id      INT PRIMARY KEY,
    email        VARCHAR(100),
    signup_date  DATE
);

CREATE TABLE user_activity (
    user_id        INT REFERENCES users(user_id),
    activity_date  DATE
);

CREATE TABLE events (
    event_id          INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id           INT REFERENCES users(user_id),
    event_type        VARCHAR(20),                  -- 'view', 'signup', 'purchase'
    event_time        TIMESTAMP,
    session_id        INT,
    session_duration  INT,                          -- seconds
    revenue           DECIMAL(10, 2),
    experiment_name   VARCHAR(50),
    variant           CHAR(1)                       -- 'A' / 'B'
);
```

DIALECTS

PostgreSQL MySQL SQL Server Oracle Db2

`GENERATED ... AS IDENTITY` is standard (PostgreSQL 10+, Oracle, Db2). MySQL uses `INT AUTO_INCREMENT PRIMARY KEY`; SQL Server uses `INT IDENTITY(1,1) PRIMARY KEY`; older PostgreSQL code often uses `SERIAL`.

SECTION 02 · PART 01: FOUNDATIONS

02 How a Query Is Processed: Logical Order of Execution

You **write** clauses in one order, but the database **evaluates** them in another. This explains most "column does not exist" errors, for example why a `SELECT` alias can't be used in `WHERE` but can be used in `ORDER BY`.

YOU WRITE

```
SELECT      -- 6
FROM        -- 1
WHERE       -- 2
GROUP BY    -- 3
HAVING      -- 4
ORDER BY    -- 8
LIMIT       -- 9
```

THE DATABASE EVALUATES

1. `FROM` and `JOIN` s: build the working set of rows
2. `WHERE` : filter individual rows (no aggregates allowed here)
3. `GROUP BY` : collapse rows into groups
4. `HAVING` : filter groups (aggregates allowed)
5. Window functions: computed over the remaining rows
6. `SELECT` : compute output expressions and aliases
7. `DISTINCT` : remove duplicate output rows
8. `ORDER BY` : sort (can use `SELECT` aliases)
9. `LIMIT` / `OFFSET` / `FETCH` : keep a slice of the sorted rows

POSTGRESQL 02.01

```postgresql
SELECT   department, AVG(salary) AS avg_salary      -- 6
FROM     employees                                  -- 1
WHERE    hire_date < DATE '2024-01-01'              -- 2
GROUP BY department                                 -- 3
HAVING   COUNT(*) >= 2                              -- 4
ORDER BY avg_salary DESC                            -- 8 (alias is allowed here)
LIMIT    3;                                         -- 9
```

SECTION 03 · PART 01: FOUNDATIONS

03 SELECT Basics

IN THIS SECTION Select columns, all columns, expressions and aliases · DISTINCT: unique values · Comments and quoting

03.1 Select columns, all columns, expressions and aliases

`SELECT` chooses the output columns. `AS` renames a column or table (an alias). Prefer listing columns over `SELECT *` in production code.

POSTGRESQL 03.01

```postgresql
-- Specific columns
SELECT first_name, last_name FROM employees;

-- All columns (fine for exploring, avoid in production)
SELECT * FROM employees;

-- Expressions and aliases
SELECT
    name,
    salary,
    salary * 12            AS annual_salary,
    salary * 1.05          AS salary_after_raise,
    'FT'                   AS contract_type     -- a literal value
FROM employees AS e;
```

03.2 DISTINCT: unique values

`DISTINCT` removes duplicate rows from the result; `COUNT(DISTINCT col)` counts unique non-NULL values.

POSTGRESQL 03.02

```postgresql
SELECT DISTINCT department FROM employees;
SELECT DISTINCT department, gender FROM employees;          -- unique combinations
SELECT COUNT(DISTINCT customer_id) AS buying_customers FROM orders;
```

DIALECTS PostgreSQL

PostgreSQL also has `SELECT DISTINCT ON (col) ...`, which keeps the first row per value of `col` (according to `ORDER BY`).

03.3 Comments and quoting

POSTGRESQL 03.03

```postgresql
-- single-line comment
/* multi-line
   comment */
SELECT 'It''s a string'      AS single_quotes_for_strings,   -- double the quote to escape it
       "name"                AS double_quotes_for_identifiers
FROM employees;
```

DIALECTS MySQL SQL Server

MySQL quotes identifiers with backticks by default; SQL Server uses square brackets (double quotes also work in both when ANSI mode is on).

SECTION 04 · PART 01: FOUNDATIONS

04 Filtering Rows (WHERE)

IN THIS SECTION Comparison operators · AND, OR, NOT and precedence · BETWEEN, IN · Pattern matching with LIKE · NULL handling in filters

04.1 Comparison operators

`=`, `<>` (standard "not equal"; `!=` also works almost everywhere), `<`, `<=`, `>`, `>=`.

POSTGRESQL 04.01

```sql
SELECT * FROM employees WHERE department = 'Sales';
SELECT * FROM employees WHERE salary > 100000;
SELECT * FROM employees WHERE department <> 'HR';
```

04.2 AND, OR, NOT and precedence

`NOT` binds tightest, then `AND`, then `OR`. Use parentheses whenever you mix `AND` and `OR`.

POSTGRESQL 04.02

```sql
-- Means: (Sales AND salary > 50000) OR anyone in Engineering
SELECT * FROM employees
WHERE department = 'Sales' AND salary > 50000
   OR department = 'Engineering';

-- Usually what you actually want:
SELECT * FROM employees
WHERE salary > 50000
  AND (department = 'Sales' OR department = 'Engineering');

SELECT * FROM employees WHERE NOT (department = 'HR');
```

04.3 BETWEEN, IN

`BETWEEN a AND b` is **inclusive** on both ends. `IN (...)` is shorthand for several `OR`ed equality tests.

POSTGRESQL 04.03

```sql
SELECT * FROM orders WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';
SELECT * FROM employees WHERE id IN (1, 2, 3, 4);
SELECT * FROM employees WHERE department IN ('Sales', 'Marketing', 'Finance');
SELECT * FROM employees WHERE department NOT IN ('HR');
```

> **WATCH OUT**
>
> for `TIMESTAMP` columns, `BETWEEN '2023-01-01' AND '2023-12-31'` misses everything after midnight on 31 December. Use a half-open range instead: `ts >= '2023-01-01' AND ts < '2024-01-01'`.

04.4 Pattern matching with LIKE

`%` matches any number of characters, `_` matches exactly one.

POSTGRESQL 04.04

```sql
SELECT * FROM employees WHERE name LIKE 'A%';       -- starts with A
SELECT * FROM employees WHERE name LIKE '%ge';      -- ends with ge
SELECT * FROM employees WHERE name LIKE '_a%';      -- second letter is a
SELECT * FROM employees WHERE email LIKE '%@corp.com';
SELECT * FROM employees WHERE name ILIKE 'amina%'; -- case-insensitive (PostgreSQL)
```

DIALECTS PostgreSQL MySQL SQL Server

`ILIKE` is PostgreSQL-only. In MySQL and SQL Server, `LIKE` is usually already case-insensitive (it depends on the collation); portable code uses `LOWER(name) LIKE 'amina%'`.

04.5 NULL handling in filters

`NULL` means "unknown". Any comparison with `NULL` (even `NULL = NULL`) is unknown, so it never passes a `WHERE`. Always use `IS NULL` / `IS NOT NULL`.

POSTGRESQL 04.05

```sql
SELECT * FROM employees WHERE email IS NULL;
SELECT * FROM employees WHERE email IS NOT NULL;

-- Trap: this returns NO rows, because gender = NULL is never true
SELECT * FROM employees WHERE gender = NULL;

-- Trap: NOT IN with a NULL in the list returns no rows
SELECT * FROM employees WHERE id NOT IN (1, 2, NULL);

-- NULL-safe comparison (standard; PostgreSQL, SQL Server 2022+)
SELECT * FROM employees WHERE gender IS DISTINCT FROM 'M';   -- includes NULL genders
```

DIALECTS MySQL

MySQL's NULL-safe equality operator is `a <=> b`.

SECTION 05 · PART 01: FOUNDATIONS

05 Sorting and Limiting

IN THIS SECTION ORDER BY · LIMIT, OFFSET and FETCH

05.1 ORDER BY

Sort ascending (`ASC` , the default) or descending (`DESC`). Sort on several columns, on aliases, or on expressions.

POSTGRESQL 05.01

```sql
SELECT name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;

-- Control where NULLs go (PostgreSQL, Oracle; standard SQL)
SELECT name, bonus FROM employees ORDER BY bonus DESC NULLS LAST;
```

DIALECTS PostgreSQL MySQL SQL Server

PostgreSQL sorts NULLs as the largest values (last in `ASC`); MySQL and SQL Server sort them first in `ASC` . MySQL/SQL Server have no `NULLS LAST` ; use `ORDER BY bonus IS NULL, bonus DESC` (MySQL) or `ORDER BY CASE WHEN bonus IS NULL THEN 1 ELSE 0 END, bonus DESC` .

05.2 LIMIT, OFFSET and FETCH

POSTGRESQL 05.02

```sql
SELECT * FROM orders ORDER BY order_date DESC LIMIT 10;
SELECT * FROM orders ORDER BY id LIMIT 10 OFFSET 20;          -- page 3 when pages hold 10 rows

-- Standard SQL form (PostgreSQL, Oracle 12c+, SQL Server 2012+ with OFFSET)
SELECT * FROM orders ORDER BY id OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

SQL SERVER 05.03

```sql
-- SQL Server
SELECT TOP 10 * FROM orders ORDER BY order_date DESC;
```

> **TIP**
>
> always pair `LIMIT` with `ORDER BY`, otherwise "the first 10 rows" is arbitrary. For deep pagination, keyset pagination is much faster than a large `OFFSET`:

POSTGRESQL 05.04

```
-- Keyset pagination: "next 10 orders after the last id I showed"
SELECT * FROM orders WHERE id > 10 ORDER BY id LIMIT 10;
```

02

Aggregation & Logic



06 **Aggregation, GROUP BY and HAVING** 16

Aggregate functions · Median and percentiles · GROUP BY · HAVING: filter groups · Conditional aggregation and FILTER · ROLLUP, CUBE and GROUPING SETS: subtotals · String aggregation

07 **Conditional Logic: CASE, COALESCE, NULLIF** 19

CASE expression · COALESCE, NULLIF, GREATEST, LEAST

SECTION 06 · PART 02: AGGREGATION & LOGIC

06 Aggregation, GROUP BY and HAVING

IN THIS SECTION Aggregate functions · Median and percentiles · GROUP BY · HAVING: filter groups · Conditional aggregation and FILTER · ROLLUP, CUBE and GROUPING SETS: subtotals · String aggregation

06.1 Aggregate functions

Aggregates collapse many rows into one value. All of them except `COUNT(*)` **ignore NULLs**.

POSTGRESQL 06.01

```sql
SELECT
    COUNT(*)                    AS total_rows,        -- counts rows
    COUNT(email)                AS rows_with_email,   -- counts non-NULL values
    COUNT(DISTINCT department)  AS departments,
    SUM(salary)                 AS total_payroll,
    AVG(salary)                 AS average_salary,
    MIN(hire_date)              AS earliest_hire,
    MAX(hire_date)              AS latest_hire,
    STDDEV(salary)              AS salary_std_dev,     -- sample standard deviation
    VARIANCE(salary)            AS salary_variance     -- sample variance
FROM employees;
```

DIALECTS PostgreSQL MySQL SQL Server

SQL Server spells these `STDEV` / `VAR` (sample) and `STDEVP` / `VARP` (population). The standard names `STDDEV_SAMP`, `STDDEV_POP`, `VAR_SAMP`, `VAR_POP` work in PostgreSQL and MySQL. `AVG` of an integer column returns an integer in SQL Server; cast first: `AVG(CAST(x AS DECIMAL(10,2)))`.

06.2 Median and percentiles

POSTGRESQL 06.02

```sql
SELECT
    PERCENTILE_CONT(0.5)  WITHIN GROUP (ORDER BY salary) AS median_salary,   -- interpolated
    PERCENTILE_DISC(0.25) WITHIN GROUP (ORDER BY salary) AS first_quartile,  -- an actual value
    PERCENTILE_CONT(0.9)  WITHIN GROUP (ORDER BY salary) AS p90_salary
FROM employees;
```

DIALECTS PostgreSQL MySQL SQL Server Oracle

PostgreSQL and Oracle use these as ordinary aggregates. SQL Server only has them as window functions: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) OVER ()`. MySQL has no percentile function; use `ROW_NUMBER()` and `COUNT(*)` to find the middle row.

06.3 GROUP BY

Produces one row per group. Every selected column must either be in `GROUP BY` or be wrapped in an aggregate.

POSTGRESQL 06.03

```postgresql
-- Average salary per department
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;

-- Several grouping columns
SELECT EXTRACT(YEAR FROM order_date)  AS yr,
       EXTRACT(MONTH FROM order_date) AS mon,
       status,
       SUM(total)                     AS total_sales
FROM orders
GROUP BY EXTRACT(YEAR FROM order_date), EXTRACT(MONTH FROM order_date), status
ORDER BY yr, mon;
```

06.4 HAVING: filter groups

`WHERE` filters rows **before** grouping; `HAVING` filters groups **after** aggregation.

POSTGRESQL 06.04

```postgresql
SELECT department, AVG(salary) AS avg_salary, COUNT(*) AS headcount
FROM employees
WHERE hire_date < DATE '2024-01-01'          -- row filter
GROUP BY department
HAVING AVG(salary) > 50000 AND COUNT(*) >= 2;  -- group filter
```

06.5 Conditional aggregation and FILTER

Count or sum only the rows that match a condition, inside one query. This is also the portable way to **pivot** rows into columns.

POSTGRESQL 06.05

```postgresql
-- Portable: CASE inside the aggregate
SELECT
    department,
    COUNT(CASE WHEN gender = 'M' THEN 1 END)                AS male_count,
    COUNT(CASE WHEN gender = 'F' THEN 1 END)                AS female_count,
    COUNT(CASE WHEN gender IS NULL THEN 1 END)              AS unknown_count,
    AVG(CASE WHEN experience_years > 5 THEN salary END)     AS experienced_avg_salary
FROM employees
GROUP BY department;

-- Standard FILTER clause (PostgreSQL, SQLite): same result, cleaner
SELECT
    department,
    COUNT(*) FILTER (WHERE gender = 'M')  AS male_count,
    COUNT(*) FILTER (WHERE gender = 'F')  AS female_count
FROM employees
GROUP BY department;
```

06.6 ROLLUP, CUBE and GROUPING SETS: subtotals

POSTGRESQL 06.06

```postgresql
-- Subtotals per year, plus a grand total row (NULLs mark the subtotal rows)
SELECT EXTRACT(YEAR FROM order_date) AS yr,
       status,
       SUM(total) AS total_sales
FROM orders
GROUP BY ROLLUP (EXTRACT(YEAR FROM order_date), status)
ORDER BY yr, status;

-- Every combination of the listed columns
SELECT department, gender, COUNT(*) FROM employees GROUP BY CUBE (department, gender);

-- Exactly the groupings you list
SELECT department, gender, COUNT(*)
FROM employees
GROUP BY GROUPING SETS ((department), (gender), ());
```

DIALECTS MySQL

MySQL writes `GROUP BY yr, status WITH ROLLUP` and has no `CUBE` or `GROUPING SETS`.

06.7 String aggregation

POSTGRESQL 06.07

```postgresql
SELECT department,
       STRING_AGG(last_name, ', ' ORDER BY last_name) AS employees
FROM employees
GROUP BY department;
```

MYSQL 06.08

```mysql
-- MySQL
SELECT department,
       GROUP_CONCAT(DISTINCT last_name ORDER BY last_name SEPARATOR ', ') AS employees
FROM employees
GROUP BY department;
```

DIALECTS SQL Server

SQL Server 2017+: `STRING_AGG(last_name, ', ') WITHIN GROUP (ORDER BY last_name)`.

SECTION 07 · PART 02: AGGREGATION & LOGIC

07 Conditional Logic: CASE, COALESCE, NULLIF

IN THIS SECTION CASE expression · COALESCE, NULLIF, GREATEST, LEAST

07.1 CASE expression

`CASE` is SQL's if/else. It works anywhere an expression is allowed (SELECT, WHERE, ORDER BY, GROUP BY, inside aggregates). Conditions are checked top to bottom and the first match wins; without `ELSE`, unmatched rows get `NULL`.

POSTGRESQL 07.01

```sql
-- Searched CASE
SELECT
    name,
    salary,
    CASE
        WHEN salary < 30000 THEN 'Entry Level'
        WHEN salary BETWEEN 30000 AND 60000 THEN 'Mid Level'
        WHEN salary BETWEEN 60000.01 AND 100000 THEN 'Senior'
        ELSE 'Executive'
    END AS level
FROM employees;

-- Simple CASE (compares one expression to values)
SELECT name,
       CASE gender WHEN 'M' THEN 'Male' WHEN 'F' THEN 'Female' ELSE 'Not stated' END AS gender_label
FROM employees;

-- Nested CASE
SELECT
    product_name,
    units_sold,
    CASE
        WHEN units_sold > 1000 THEN
            CASE
                WHEN profit_margin > 0.3 THEN 'High Sales, High Margin'
                ELSE 'High Sales, Low Margin'
            END
        ELSE 'Low Sales'
    END AS performance_category
FROM products;

-- CASE in ORDER BY: custom sort order
SELECT * FROM orders
ORDER BY CASE status WHEN 'paid' THEN 1 WHEN 'shipped' THEN 2 ELSE 3 END, order_date;
```

07.2 COALESCE, NULLIF, GREATEST, LEAST

- `COALESCE(a, b, c)` returns the first non-NULL argument.
- `NULLIF(a, b)` returns NULL when `a = b`, otherwise `a`. Its classic use is avoiding division by zero.
- `GREATEST` / `LEAST` return the largest or smallest of their arguments.

POSTGRESQL 07.02

```sql
SELECT
    name,
    COALESCE(bonus, 0)                      AS bonus,           -- replace NULL with 0
    COALESCE(email, phone, 'No contact')    AS contact_info,    -- first available value
    NULLIF(salary, 0)                       AS salary_no_zero,  -- 0 becomes NULL
    bonus / NULLIF(salary, 0)               AS bonus_ratio,     -- no divide-by-zero error
    GREATEST(salary, 30000)                 AS salary_floor_applied,
    LEAST(experience_years, 10)             AS capped_experience
FROM employees;
```

DIALECTS PostgreSQL MySQL SQL Server

SQL Server also has `ISNULL(a, b)`, MySQL has `IFNULL(a, b)` (both take two arguments only; `COALESCE` is portable). `GREATEST` / `LEAST` arrived in SQL Server 2022. In PostgreSQL they ignore NULLs; in MySQL any NULL argument makes the result NULL.

03

Combining Data



08 **Joins** 22

The join types · Joining several tables · USING and NATURAL · Semi-joins and anti-joins (EXISTS / NOT EXISTS) · Join pitfalls

09 **Subqueries and CTEs** 25

Subquery types · Common Table Expressions (CTEs) · Recursive CTEs: hierarchies and series

10 **Window Functions (Critical for Data Science)** 28

Ranking functions · Top-N per group · LAG and LEAD (time series) · Moving averages and cumulative totals · Window frame specifications · FIRST_VALUE, LAST_VALUE, NTH_VALUE and named windows

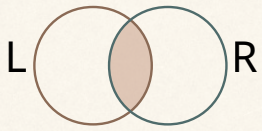
11 **Set Operations** 31

SECTION 08 · PART 03: COMBINING DATA

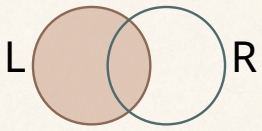
08 Joins

IN THIS SECTION The join types · Joining several tables · USING and NATURAL · Semi-joins and anti-joins (EXISTS / NOT EXISTS) · Join pitfalls

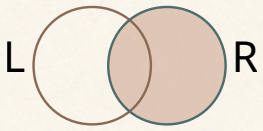
Joins combine rows from two tables using a matching condition.



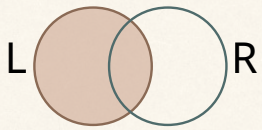
INNER JOIN: only rows that match on both sides.



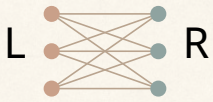
LEFT JOIN: every row from the left table, plus matches from the right (NULLs where there is no match).



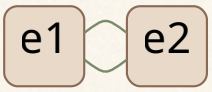
RIGHT JOIN: the mirror image of LEFT JOIN.



FULL OUTER JOIN: every row from both sides.



CROSS JOIN: every combination (Cartesian product).



Self join: a table joined to itself.

08.1 The join types

POSTGRESQL 08.01

```postgresql
-- INNER JOIN: orders with their customer
SELECT o.id, o.order_date, c.name AS customer
FROM orders o
INNER JOIN customers c ON c.id = o.customer_id;

-- LEFT JOIN: all customers, with their orders if any
SELECT c.name, o.order_date, o.total
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id;

-- RIGHT JOIN: all customers again, written the other way round
SELECT o.*, c.name
FROM orders o
RIGHT JOIN customers c ON o.customer_id = c.id;

-- FULL OUTER JOIN: all products and all categories, matched where possible
SELECT p.product_name, cat.category_name
FROM products p
FULL OUTER JOIN categories cat ON p.category_id = cat.id;
-- CROSS JOIN: every department paired with every category
SELECT d.name, cat.category_name
FROM departments d CROSS JOIN categories cat;

-- SELF JOIN: each employee with their manager
SELECT e1.name AS employee, e2.name AS manager
FROM employees e1
LEFT JOIN employees e2 ON e1.manager_id = e2.id;
```

DIALECTS MySQL

MySQL has no `FULL OUTER JOIN`; emulate it with a `LEFT JOIN UNION` a `RIGHT JOIN`.

08.2 Joining several tables

POSTGRESQL 08.02

```postgresql
SELECT
    c.name          AS customer,
    o.order_date,
    p.product_name,
    cat.category_name
FROM customers c
JOIN orders o           ON c.id = o.customer_id
JOIN order_items oi     ON o.id = oi.order_id
JOIN products p         ON oi.product_id = p.id
LEFT JOIN categories cat ON p.category_id = cat.id
WHERE o.order_date >= '2023-01-01';
```

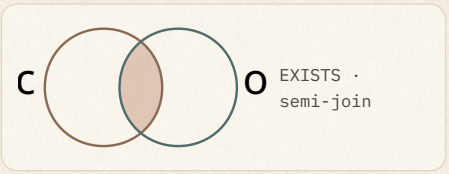
08.3 USING and NATURAL

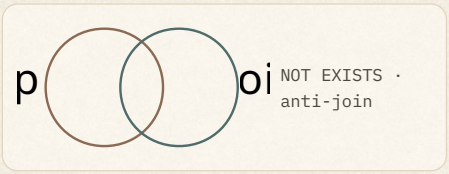
`USING (col)` is shorthand when both tables share the column name. Avoid `NATURAL JOIN`: it silently joins on every same-named column.

POSTGRESQL 08.03

```
SELECT user_id, u.email, a.activity_date
FROM users u
JOIN user_activity a USING (user_id);
```

08.4 Semi-joins and anti-joins (EXISTS / NOT EXISTS)





POSTGRESQL 08.04

```
-- Customers who HAVE ordered (semi-join, no duplicate customers)
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- Products that have NEVER been ordered (anti-join)
SELECT p.* FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.id);

-- Same anti-join with LEFT JOIN ... IS NULL
SELECT p.*
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.id
WHERE oi.id IS NULL;
```

08.5 Join pitfalls

- **Filtering a LEFT JOIN in WHERE turns it into an INNER JOIN.** Put conditions on the right-hand table in the `ON` clause.
- **Fan-out:** joining a one-to-many relationship repeats the “one” side; aggregating after the join can double count. Aggregate in a subquery or CTE first.

POSTGRESQL 08.05

```
-- Wrong: customers with no 2024 orders disappear
SELECT c.name, o.id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.order_date >= '2024-01-01';

-- Right: keeps every customer, only 2024 orders attached
SELECT c.name, o.id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND o.order_date >= '2024-01-01';
```

SECTION 09 · PART 03: COMBINING DATA

09 Subqueries and CTEs

IN THIS SECTION Subquery types · Common Table Expressions (CTEs) · Recursive CTEs: hierarchies and series

09.1 Subquery types

A subquery is a query nested inside another. It can return a single value (scalar), a list (used with `IN`), or a whole table (used in `FROM`, a "derived table" that needs an alias).

POSTGRESQL 09.01

```postgresql
-- Scalar subquery in WHERE: products priced above average
SELECT product_name, price FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- Scalar subquery in SELECT
SELECT product_name, price,
       (SELECT AVG(price) FROM products) AS avg_price
FROM products;

-- List subquery with IN
SELECT name FROM customers
WHERE id IN (SELECT customer_id FROM orders WHERE total > 50000);

-- Derived table (subquery in FROM) - the alias is required
SELECT dept_stats.department, dept_stats.avg_salary
FROM (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) AS dept_stats
WHERE dept_stats.avg_salary > 60000;

-- Correlated subquery: re-evaluated for each outer row
SELECT e1.name, e1.salary, e1.department
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);

-- EXISTS: customers with at least one order over 50,000
SELECT c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
```

```
    FROM orders o
    WHERE o.customer_id = c.id
      AND o.total > 50000
);
```

> TIP
>
> prefer `NOT EXISTS` to `NOT IN (subquery)` ; if the subquery returns any NULL, `NOT IN` returns no rows at all.

09.2 Common Table Expressions (CTEs)

`WITH name AS (...)` defines a named, temporary result set for one statement. Later CTEs can reference earlier ones. CTEs make long queries readable step by step.

POSTGRESQL 09.02

```
WITH
    department_stats AS (
        SELECT department, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY department
    ),
    high_paying_depts AS (
        SELECT department
        FROM department_stats
        WHERE avg_salary > 70000
    )
SELECT e.name, e.department, e.salary, d.avg_salary
FROM employees e
JOIN department_stats d ON e.department = d.department
WHERE e.department IN (SELECT department FROM high_paying_depts);
```

A longer multi-step example (customer lifetime value and segments):

POSTGRESQL 09.03

```
WITH
    -- First CTE
    customer_stats AS (
        SELECT
            customer_id,
            COUNT(*)        AS order_count,
            SUM(total)      AS lifetime_value,
            MIN(order_date) AS first_order,
            MAX(order_date) AS last_order
        FROM orders
        GROUP BY customer_id
    ),
    -- Second CTE can reference the first
    active_customers AS (
        SELECT customer_id
        FROM customer_stats
        WHERE last_order >= CURRENT_DATE - INTERVAL '30 days'
          AND order_count >= 3
    ),
    -- Third CTE
```

```
    customer_segments AS (
        SELECT
            cs.*,
            CASE
                WHEN lifetime_value > 100000 THEN 'VIP'
                WHEN lifetime_value BETWEEN 10000 AND 100000 THEN 'Regular'
                ELSE 'Casual'
            END AS segment
        FROM customer_stats cs
    )
-- Main query using the CTEs
SELECT
    c.name,
    cs.segment,
    cs.lifetime_value,
    cs.order_count
FROM customers c
JOIN customer_segments cs ON c.id = cs.customer_id
WHERE c.id IN (SELECT customer_id FROM active_customers)
ORDER BY cs.lifetime_value DESC;
```

09.3 Recursive CTEs: hierarchies and series

A recursive CTE has an anchor query, `UNION ALL`, and a recursive part that references the CTE itself. Typical uses are org charts, category trees and generating series.

POSTGRESQL 09.04

```
-- Org chart: everyone under the CTO, with their level
WITH RECURSIVE org AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL AND title = 'CTO'          -- anchor
    UNION ALL
    SELECT e.id, e.name, e.manager_id, org.level + 1
    FROM employees e
    JOIN org ON e.manager_id = org.id                   -- recursive step
)
SELECT * FROM org ORDER BY level, name;

-- Generate the numbers 1 to 10
WITH RECURSIVE n AS (
    SELECT 1 AS x
    UNION ALL
    SELECT x + 1 FROM n WHERE x < 10
)
SELECT x FROM n;
```

DIALECTS MySQL SQL Server

SQL Server omits the word `RECURSIVE` (`WITH org AS (...)`). MySQL 8.0+ supports `WITH RECURSIVE`.

SECTION 10 · PART 03: COMBINING DATA

10 Window Functions (Critical for Data Science)

IN THIS SECTION Ranking functions · Top-N per group · LAG and LEAD (time series) · Moving averages and cumulative totals · Window frame specifications · FIRST_VALUE, LAST_VALUE, NTH_VALUE and named windows

A window function computes a value **across a set of related rows without collapsing them**, unlike `GROUP BY`.
Syntax: `function() OVER (PARTITION BY ... ORDER BY ... frame)`.

`function() OVER ( PARTITION BY ... ORDER BY ... frame )`

`PARTITION BY`: restart the calculation for each group.

`ORDER BY`: the order of rows inside the window (needed for ranking, running totals, LAG/LEAD).

Frame (`ROWS BETWEEN ...`): which rows around the current one to include.

10.1 Ranking functions

POSTGRESQL 10.01

```sql
SELECT
    name,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,     -- 1,2,3,4 (ties broken arbitrarily)
    RANK()       OVER (ORDER BY salary DESC) AS rank_gaps,   -- 1,2,2,4
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rank_dense,  -- 1,2,2,3
    NTILE(4)     OVER (ORDER BY salary DESC) AS quartile,    -- bucket 1..4
    PERCENT_RANK() OVER (ORDER BY salary)    AS pct_rank,    -- 0..1
    CUME_DIST()    OVER (ORDER BY salary)    AS cume_dist
FROM employees;

-- Partitioned ranking: rank within each department
SELECT
    department,
    name,
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;
```

NOTE

don't name a column `rank` or `row_number`; they are reserved words in MySQL 8 and confusing everywhere.

salary DESC		ROW_NUMBER	RANK	DENSE_RANK
		1	1	1
	tie	2	2	2
	tie	3	2	2
		4	4	3

Ties as described in the comments above: `ROW_NUMBER` *1,2,3,4 ·* `RANK` *1,2,2,4 ·* `DENSE_RANK` *1,2,2,3*

10.2 Top-N per group

Window functions cannot appear in `WHERE` (they run after it), so filter in an outer query.

POSTGRESQL 10.02

```postgresql
-- Two highest-paid employees in each department
SELECT *
FROM (
    SELECT department, name, salary,
           ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
    FROM employees
) ranked
WHERE rn <= 2;
```

10.3 LAG and LEAD (time series)

`LAG` reads a value from a previous row and `LEAD` from a following row, which is ideal for day-over-day changes.

POSTGRESQL 10.03

```postgresql
SELECT
    sale_date,
    revenue,
    LAG(revenue, 1)  OVER (ORDER BY sale_date) AS previous_day,
    LEAD(revenue, 1) OVER (ORDER BY sale_date) AS next_day,
    revenue - LAG(revenue, 1) OVER (ORDER BY sale_date) AS daily_change,
    ROUND(
        (revenue - LAG(revenue, 1) OVER (ORDER BY sale_date)) * 100.0
        / NULLIF(LAG(revenue, 1) OVER (ORDER BY sale_date), 0), 2
    ) AS pct_change
FROM daily_sales;
```

10.4 Moving averages and cumulative totals

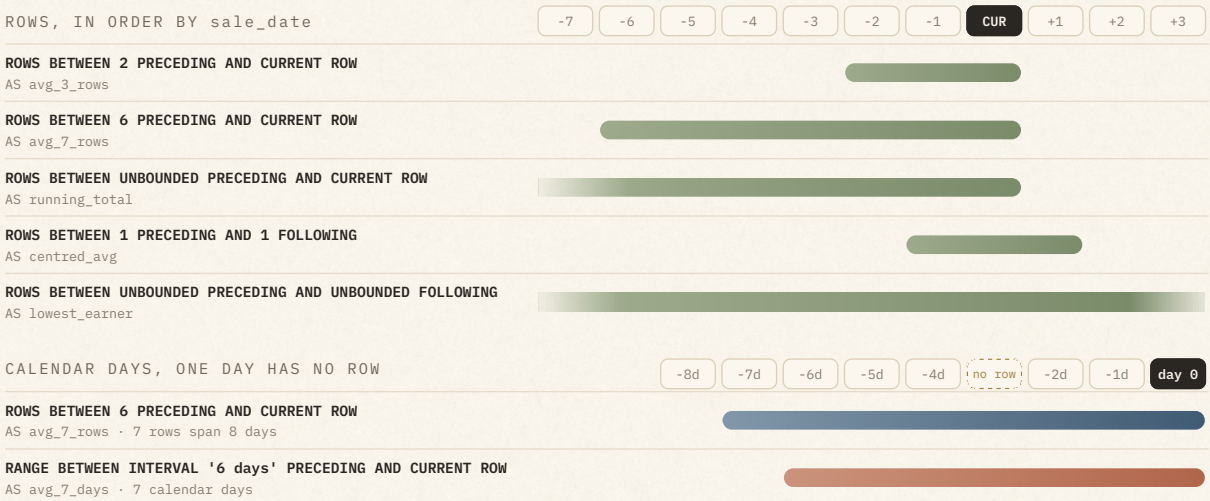
POSTGRESQL 10.04

```sql
SELECT
    sale_date,
    revenue,
    AVG(revenue) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)  AS weekly_avg,
    AVG(revenue) OVER (ORDER BY sale_date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW) AS monthly_avg,
    SUM(revenue) OVER (ORDER BY sale_date)                                           AS cumulative_revenue,
    COUNT(*)     OVER (ORDER BY sale_date)                                           AS cumulative_count,
    revenue * 100.0 / SUM(revenue) OVER ()                                           AS pct_of_total
FROM daily_sales;
```

10.5 Window frame specifications

`ROWS` counts physical rows; `RANGE` uses values (so ties and date gaps are handled by value). With `ORDER BY` and no frame, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, a running total.

FIGURE *Which rows each frame includes, relative to the current row*



POSTGRESQL 10.05

```sql
SELECT
    sale_date,
    revenue,
    -- Last 3 rows including current
    AVG(revenue) OVER (ORDER BY sale_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS avg_3_rows,
    -- Last 7 ROWS (equals 7 days only if there is exactly one row per day)
    AVG(revenue) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS avg_7_rows,
    -- Last 7 calendar DAYS, correct even with missing days (PostgreSQL 11+)
    AVG(revenue) OVER (ORDER BY sale_date
                       RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW) AS avg_7_days,
    -- All previous rows
    SUM(revenue) OVER (ORDER BY sale_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
    -- Centred window: previous, current, next
    AVG(revenue) OVER (ORDER BY sale_date ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS centred_avg
FROM daily_sales;
```

10.6 FIRST_VALUE, LAST_VALUE, NTH_VALUE and named windows

POSTGRESQL 10.06

```sql
SELECT
    department,
    name,
    salary,
    FIRST_VALUE(name) OVER w AS top_earner,
    LAST_VALUE(name)  OVER (w ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS lowest_earner,
    AVG(salary)       OVER (PARTITION BY department) AS avg_dept_salary
FROM employees
WINDOW w AS (PARTITION BY department ORDER BY salary DESC);
```

WATCH OUT

`LAST_VALUE` with the default frame only looks up to the current row, so it returns the current row. Extend the frame to `UNBOUNDED FOLLOWING` as shown.

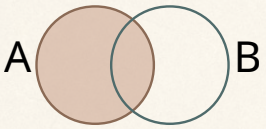
DIALECTS PostgreSQL MySQL SQL Server

the `WINDOW` clause works in PostgreSQL, MySQL 8.0+ and SQL Server 2022+. `RANGE` with an `INTERVAL` offset is not supported in SQL Server.

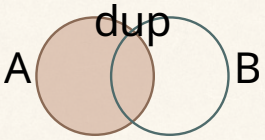
SECTION II · PART 03: COMBINING DATA

11 Set Operations

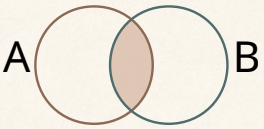
Combine the results of two queries that have the same number of columns with compatible types. Column names come from the first query.



UNION: rows from both, duplicates removed (slower: it has to de-duplicate).



UNION ALL: rows from both, duplicates kept (use it when you know there are none, or want them).



INTERSECT: rows present in both.



EXCEPT: rows in the first but not the second.

POSTGRESQL 11.01

```postgresql
-- Every city that has a customer or a department office
SELECT city FROM customers
UNION
SELECT location FROM departments WHERE location IS NOT NULL;

-- Keep duplicates
SELECT city FROM customers
UNION ALL
SELECT location FROM departments;

-- Cities with both a customer and an office
SELECT city FROM customers
INTERSECT
SELECT location FROM departments;

-- Cities with customers but no office
SELECT city FROM customers
EXCEPT
SELECT location FROM departments WHERE location IS NOT NULL;
```

DIALECTS MySQL Oracle

Oracle calls `EXCEPT MINUS`. MySQL supports `INTERSECT` and `EXCEPT` from 8.0.31. To sort the combined result, put one `ORDER BY` at the very end.

04

Defining & Changing Data

12 **Data Definition (DDL): CREATE, ALTER, DROP** 34

Common data types · CREATE TABLE · Create a table from a query, temporary tables · ALTER TABLE · DROP and TRUNCATE

13 **Constraints and Keys** 37

14 **Data Manipulation (DML): INSERT, UPDATE, DELETE** 38

INSERT · UPSERT: insert or update if the row exists · UPDATE · DELETE

15 **Indexes** 41

16 **Views and Materialized Views** 42

17 **Transactions** 43

SECTION 12 · PART 04: DEFINING & CHANGING DATA

12 Data Definition (DDL): CREATE, ALTER, DROP

IN THIS SECTION Common data types · CREATE TABLE · Create a table from a query, temporary tables · ALTER TABLE · DROP and TRUNCATE

12.1 Common data types

- `INT` / `INTEGER`, `BIGINT`, `SMALLINT`: whole numbers
- `DECIMAL(p, s)` / `NUMERIC(p, s)`: exact decimals, use them for money (p = total digits, s = digits after the point)
- `REAL`, `FLOAT`, `DOUBLE PRECISION`: approximate numbers (fine for measurements, never for money)
- `CHAR(n)`: fixed length; `VARCHAR(n)`: variable length up to n; `TEXT`: long text
- `DATE`, `TIME`, `TIMESTAMP` (date + time), `TIMESTAMP WITH TIME ZONE`, `INTERVAL`
- `BOOLEAN`: true/false
- `JSON`, `JSONB` (binary, indexable, PostgreSQL), `UUID`, arrays (`INT[]`, PostgreSQL)

> **DIALECTS** MySQL · SQL Server
>
> MySQL has `BOOLEAN` as an alias for `TINYINT(1)` and uses `DATETIME` / `TIMESTAMP`; SQL Server uses `BIT` for booleans, `DATETIME2` for timestamps, `NVARCHAR` for Unicode text, `UNIQUEIDENTIFIER` for UUIDs.

12.2 CREATE TABLE

POSTGRESQL 12.01

```sql
CREATE TABLE staff (
    id             INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name     VARCHAR(50)  NOT NULL,
    last_name      VARCHAR(50)  NOT NULL,
    email          VARCHAR(100) UNIQUE,
    salary         DECIMAL(10, 2) DEFAULT 0,
    hire_date      DATE DEFAULT CURRENT_DATE,
    department_id  INT REFERENCES departments(id),
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CHECK (salary >= 0)
);

-- Only create it if it doesn't exist yet
CREATE TABLE IF NOT EXISTS staff (id INT PRIMARY KEY);
```

MYSQL 12.02

```mysql
-- MySQL
CREATE TABLE staff (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    first_name    VARCHAR(50) NOT NULL,
    salary        DECIMAL(10, 2) DEFAULT 0,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

SQL SERVER 12.03

```sql
-- SQL Server
CREATE TABLE staff (
    id          INT IDENTITY(1,1) PRIMARY KEY,
    first_name  NVARCHAR(50) NOT NULL,
    salary      DECIMAL(10, 2) DEFAULT 0,
    created_at  DATETIME2 DEFAULT SYSDATETIME()
);
```

12.3 Create a table from a query, temporary tables

POSTGRESQL 12.04

```postgresql
-- Copy structure + data from a query
CREATE TABLE high_earners AS
SELECT * FROM employees WHERE salary > 100000;

-- Copy structure only (PostgreSQL)
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);

-- Temporary table: dropped automatically at the end of the session
CREATE TEMP TABLE session_data (
    user_id     INT,
    session_id  UUID,
    activity    JSONB
);
```

DIALECTS MySQL SQL Server

SQL Server uses `SELECT * INTO high_earners FROM employees WHERE ...` and `#session_data` for temp tables.
MySQL: `CREATE TEMPORARY TABLE`, and `CREATE TABLE orders_archive LIKE orders`.

12.4 ALTER TABLE

POSTGRESQL 12.05

```postgresql
-- Add a column
ALTER TABLE employees ADD COLUMN middle_name VARCHAR(50);

-- Rename a column / a table
ALTER TABLE employees RENAME COLUMN middle_name TO second_name;
ALTER TABLE customers RENAME TO clients;

-- Change a column's type
ALTER TABLE employees ALTER COLUMN email TYPE VARCHAR(150);

-- Set / drop a default and NOT NULL
ALTER TABLE employees ALTER COLUMN bonus SET DEFAULT 0;
ALTER TABLE employees ALTER COLUMN phone DROP NOT NULL;

-- Drop a column
ALTER TABLE employees DROP COLUMN second_name;
```

MYSQL 12.06

```mysql
-- MySQL
ALTER TABLE employees MODIFY COLUMN email VARCHAR(150);
ALTER TABLE employees RENAME COLUMN middle_name TO second_name;  -- MySQL 8.0+
```

SQL SERVER 12.07

```sql-server
-- SQL Server
ALTER TABLE employees ALTER COLUMN email VARCHAR(150);
EXEC sp_rename 'employees.middle_name', 'second_name', 'COLUMN';
ALTER TABLE employees ADD middle_name VARCHAR(50);              -- no COLUMN keyword
```

12.5 DROP and TRUNCATE

POSTGRESQL 12.08

```postgresql
DROP TABLE IF EXISTS high_earners;        -- removes the table and its data
DROP TABLE IF EXISTS staff CASCADE;       -- PostgreSQL: also drops dependent views/constraints
TRUNCATE TABLE daily_sales;               -- removes all rows, keeps the table
```

`TRUNCATE` vs `DELETE`: `TRUNCATE` is much faster on big tables, can't have a `WHERE`, resets identity counters (MySQL; PostgreSQL with `RESTART IDENTITY`) and fails if other tables reference the table by foreign key. In MySQL and Oracle it commits immediately and can't be rolled back; in PostgreSQL and SQL Server it can be rolled back inside a transaction.

SECTION 13 · PART 04: DEFINING & CHANGING DATA

13 Constraints and Keys

Constraints make the database reject bad data.

- `PRIMARY KEY`: unique + not null; identifies each row (one per table, can span several columns).
- `FOREIGN KEY ... REFERENCES`: the value must exist in the parent table.
- `UNIQUE`: no duplicate values (NULLs are allowed, usually several of them).
- `NOT NULL`: a value is required.
- `CHECK`: the value must satisfy a condition.
- `DEFAULT`: the value used when none is supplied (not strictly a constraint).

POSTGRESQL 13.01

```sql
CREATE TABLE projects (
    id            INT GENERATED BY DEFAULT AS IDENTITY,
    code          VARCHAR(10)  NOT NULL,
    name          VARCHAR(100) NOT NULL,
    budget        DECIMAL(12, 2) DEFAULT 0 CHECK (budget >= 0),
    start_date    DATE,
    end_date      DATE,
    department_id INT,
    CONSTRAINT pk_projects       PRIMARY KEY (id),
    CONSTRAINT uq_projects_code  UNIQUE (code),
    CONSTRAINT chk_dates         CHECK (end_date IS NULL OR end_date >= start_date),
    CONSTRAINT fk_projects_dept  FOREIGN KEY (department_id)
        REFERENCES departments(id)
        ON DELETE SET NULL           -- or CASCADE / RESTRICT / NO ACTION
        ON UPDATE CASCADE
);
-- Composite primary key (many-to-many link table)
CREATE TABLE project_members (
    project_id  INT REFERENCES projects(id) ON DELETE CASCADE,
    employee_id INT REFERENCES employees(id),
    role        VARCHAR(30),
    PRIMARY KEY (project_id, employee_id)
);

-- Add and drop constraints on an existing table
ALTER TABLE orders ADD CONSTRAINT chk_total_positive CHECK (total >= 0);
ALTER TABLE orders DROP CONSTRAINT chk_total_positive;
ALTER TABLE customers ADD CONSTRAINT uq_customers_email UNIQUE (email);
```

> **ON DELETE OPTIONS**
>
> `CASCADE` deletes child rows too, `SET NULL` clears the reference, `RESTRICT` / `NO ACTION` (the default) refuses the delete while children exist.

DIALECTS MySQL

MySQL drops these with `ALTER TABLE orders DROP CHECK chk_total_positive` and `DROP FOREIGN KEY fk_name`, and enforces `CHECK` only from 8.0.16.

SECTION 14 · PART 04: DEFINING & CHANGING DATA

14 Data Manipulation (DML): INSERT, UPDATE, DELETE

IN THIS SECTION INSERT · UPSERT: insert or update if the row exists · UPDATE · DELETE

14.1 INSERT

POSTGRESQL 14.01

```sql
-- Single row (always list the columns)
INSERT INTO customers (name, email, city, signup_date)
VALUES ('Gamma Tech', 'hi@gamma.ke', 'Nairobi', CURRENT_DATE);

-- Multiple rows
INSERT INTO categories (category_name)
VALUES
    ('Toys'),
    ('Garden'),
    ('Sports');

-- Insert from a query
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);
INSERT INTO orders_archive
SELECT * FROM orders
WHERE order_date < '2023-01-01';

-- Get generated values back (PostgreSQL, SQLite, MariaDB)
INSERT INTO customers (name, city) VALUES ('Hewa Air', 'Eldoret')
RETURNING id, name;
```

DIALECTS MySQL SQL Server

SQL Server returns values with `OUTPUT inserted.id` placed before `VALUES`; MySQL uses `SELECT LAST_INSERT_ID()`.

14.2 UPSERT: insert or update if the row exists

POSTGRESQL 14.02

```postgresql
-- PostgreSQL / SQLite
INSERT INTO daily_sales (sale_date, revenue)
VALUES (CURRENT_DATE, 999.99)
ON CONFLICT (sale_date)
DO UPDATE SET revenue = EXCLUDED.revenue;

-- Standard MERGE (PostgreSQL 15+, SQL Server, Oracle)
MERGE INTO daily_sales AS t
USING (SELECT CURRENT_DATE AS sale_date, 1234.50 AS revenue) AS s
ON t.sale_date = s.sale_date
WHEN MATCHED THEN UPDATE SET revenue = s.revenue
WHEN NOT MATCHED THEN INSERT (sale_date, revenue) VALUES (s.sale_date, s.revenue);
```

MYSQL 14.03

```mysql
-- MySQL
INSERT INTO daily_sales (sale_date, revenue)
VALUES (CURRENT_DATE, 999.99)
ON DUPLICATE KEY UPDATE revenue = VALUES(revenue);

-- MySQL 8.0.20+ preferred form (VALUES() is deprecated there)
INSERT INTO daily_sales (sale_date, revenue)
VALUES (CURRENT_DATE, 999.99) AS new
ON DUPLICATE KEY UPDATE revenue = new.revenue;
```

14.3 UPDATE

Always write the `WHERE` first. An `UPDATE` without `WHERE` changes every row. Run the same `WHERE` as a `SELECT` before updating.

POSTGRESQL 14.04

```postgresql
-- Basic update
UPDATE employees
SET salary = salary * 1.05
WHERE department = 'Engineering';

-- Update several columns
UPDATE employees
SET title  = 'Senior Analyst',
    salary = salary + 5000,
    bonus  = COALESCE(bonus, 0) + 1000
WHERE id = 8;

-- Update with a subquery
UPDATE products
SET price = price * 0.9
WHERE category_id IN (
    SELECT id
    FROM categories
    WHERE category_name = 'Clearance'
);

-- Update using another table (PostgreSQL / SQL Server style)
UPDATE employees e
SET department = d.name
FROM departments d
WHERE e.department_id = d.id;
```

MYSQL 14.05

```mysql
-- MySQL: update with a join
UPDATE employees e
JOIN departments d ON e.department_id = d.id
SET e.department = d.name;
```

14.4 DELETE

POSTGRESQL 14.06

```postgresql
-- Delete specific rows (child rows first because of the foreign key)
DELETE FROM order_items
WHERE order_id IN (SELECT id FROM orders WHERE status = 'cancelled');

DELETE FROM orders
WHERE status = 'cancelled';

-- Delete using a subquery
DELETE FROM user_activity
WHERE user_id IN (
    SELECT user_id
    FROM users
    WHERE signup_date < '2024-02-01'
);

-- Remove all rows quickly
TRUNCATE TABLE daily_sales;
```

> **TIP**
>
> in PostgreSQL, `DELETE ... RETURNING *` shows exactly what was removed.

SECTION 15 · PART 04: DEFINING & CHANGING DATA

15 Indexes

An index is a sorted lookup structure (usually a B-tree) that lets the database find rows without scanning the whole table. Indexes speed up `WHERE`, `JOIN`, `ORDER BY` and `GROUP BY` on the indexed columns, but slow down writes and use disk space.

POSTGRESQL 15.01

```postgresql
-- Single-column index on a frequently filtered column
CREATE INDEX idx_employee_dept ON employees(department);
CREATE INDEX idx_users_email ON users(email);

-- Descending index (helps ORDER BY order_date DESC)
CREATE INDEX idx_orders_date ON orders(order_date DESC);

-- Composite index: column order matters (leftmost prefix rule)
-- serves WHERE customer_id = ?  and  WHERE customer_id = ? ORDER BY order_date DESC
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date DESC);
CREATE INDEX idx_products_category_price ON products(category_id, price);
CREATE INDEX idx_dept_salary ON employees(department, salary);

-- Unique index: also enforces uniqueness
CREATE UNIQUE INDEX idx_customers_email_unique ON customers(email);

-- Expression index: makes WHERE LOWER(last_name) = 'otieno' fast
CREATE INDEX idx_name_lower ON employees(LOWER(last_name));

-- Partial index: index only the rows you query (PostgreSQL, SQLite)
CREATE INDEX idx_orders_paid ON orders(order_date) WHERE status = 'paid';

-- Remove an index
DROP INDEX idx_employee_dept;
```

DIALECTS

PostgreSQL MySQL SQL Server

MySQL needs the table name to drop: `DROP INDEX idx_employee_dept ON employees` (also SQL Server). SQL Server calls partial indexes "filtered indexes" (same `WHERE` syntax). MySQL supports expression indexes from 8.0.13 with double parentheses: `CREATE INDEX idx_name_lower ON employees ((LOWER(last_name)))`. PostgreSQL can build an index without blocking writes using `CREATE INDEX CONCURRENTLY` (not inside a transaction).

WHEN AN INDEX IS NOT USED

wrapping the column in a function (`WHERE YEAR(order_date) = 2023`), a leading wildcard (`LIKE '%son'`), implicit type conversions, or very low selectivity (for example a column with only two values). Rewrite as a range: `WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01'`.

SECTION 16 · PART 04: DEFINING & CHANGING DATA

16 Views and Materialized Views

A **view** is a saved query that behaves like a table; it stores no data and always shows current results. A **materialized view** stores the result physically, so it is fast to read but must be refreshed.

POSTGRESQL 16.01

```postgresql
-- Create a view
CREATE VIEW employee_public_info AS
SELECT id, first_name, last_name, department, title
FROM employees;

SELECT * FROM employee_public_info WHERE department = 'Sales';

-- Change it
CREATE OR REPLACE VIEW employee_public_info AS
SELECT id, first_name, last_name, department, title, hire_date
FROM employees;

-- Materialized view for an expensive summary (PostgreSQL, Oracle)
CREATE MATERIALIZED VIEW daily_order_summary AS
SELECT
    order_date      AS day,
    COUNT(*)        AS total_orders,
    SUM(total)      AS total_amount
FROM orders
GROUP BY order_date;

-- Refresh materialized view after the base data changes
REFRESH MATERIALIZED VIEW daily_order_summary;

-- Remove them
DROP MATERIALIZED VIEW daily_order_summary;
DROP VIEW employee_public_info;
```

DIALECTS MySQL SQL Server

SQL Server uses `CREATE OR ALTER VIEW` and has “indexed views” instead of materialized views. MySQL has no materialized views (use a summary table refreshed by a scheduled event).

SECTION 17 · PART 04: DEFINING & CHANGING DATA

17 Transactions

A transaction groups statements so they succeed or fail **together** (ACID: Atomic, Consistent, Isolated, Durable). If anything goes wrong, `ROLLBACK` undoes every change since `BEGIN`.

POSTGRESQL 17.01

```postgresql
BEGIN;                                          -- start a transaction

INSERT INTO orders (customer_id, order_date, total, status)
VALUES (1, CURRENT_DATE, 4500, 'paid');

UPDATE products
SET units_sold = units_sold + 1
WHERE id = 3;

COMMIT;                                         -- make both changes permanent
```

POSTGRESQL 17.02

```postgresql
BEGIN;
DELETE FROM order_items;                        -- oops
ROLLBACK;                                       -- nothing was deleted

-- Savepoints: undo part of a transaction
BEGIN;
UPDATE employees SET salary = salary + 1000 WHERE id = 3;
SAVEPOINT after_raise;
UPDATE employees SET salary = 0 WHERE id = 3;   -- mistake
ROLLBACK TO SAVEPOINT after_raise;              -- undo only the mistake
COMMIT;                                         -- keeps the +1000 raise

-- Isolation level for one transaction
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT SUM(total) FROM orders;
COMMIT;
```

MYSQL 17.03

```mysql
-- MySQL
START TRANSACTION;
UPDATE products SET units_sold = units_sold + 1 WHERE id = 3;
COMMIT;
```

SQL SERVER 17.04

```sql-server
-- SQL Server
BEGIN TRANSACTION;
UPDATE products SET units_sold = units_sold + 1 WHERE id = 3;
COMMIT TRANSACTION;   -- or ROLLBACK TRANSACTION
```

Isolation levels (weakest to strongest): `READ UNCOMMITTED`, `READ COMMITTED` (the default in PostgreSQL and SQL Server), `REPEATABLE READ` (the default in MySQL InnoDB), `SERIALIZABLE`. Stronger levels prevent more anomalies (dirty reads, non-repeatable reads, phantom rows) at the cost of more blocking or retries.

> DIALECTS PostgreSQL MySQL SQL Server Oracle
>
> PostgreSQL and SQL Server can roll back DDL (`CREATE`, `ALTER`, `DROP`); MySQL and Oracle commit DDL immediately. Most clients run in autocommit mode: each statement is its own transaction unless you start one.

05

Functions & Patterns



18 **Common Functions** 47

String functions · Pattern matching with regular expressions · Date and time: current values and extraction · Date arithmetic and truncation · Date ranges and comparisons · Numeric functions · Type conversion (casting) · NULL handling summary

19 **Real-World Data Science Patterns** 52

Remove duplicates (keep the latest row per key) · Customer segmentation (spend quartile and recency) · Customer cohort analysis (monthly retention) · RFM analysis (Recency, Frequency, Monetary) · Time-series feature engineering · A/B test analysis · Running totals, share of total, gaps

SECTION 18 · PART 05: FUNCTIONS & PATTERNS

18 Common Functions

IN THIS SECTION String functions · Pattern matching with regular expressions · Date and time: current values and extraction · Date arithmetic and truncation · Date ranges and comparisons · Numeric functions · Type conversion (casting) · NULL handling summary

18.1 String functions

POSTGRESQL 18.01

```sql
SELECT
    name,
    UPPER(name)                                 AS uppercase_name,
    LOWER(name)                                 AS lowercase_name,
    INITCAP(LOWER(name))                        AS proper_case,      -- PostgreSQL, Oracle
    LENGTH(name)                                AS name_length,
    CONCAT(first_name, ' ', last_name)          AS full_name,
    first_name || ' ' || last_name              AS full_name_alt,    -- standard; NULL if any part is NULL
    SUBSTRING(email FROM 1 FOR POSITION('@' IN email) - 1) AS username,
    SPLIT_PART(email, '@', 2)                   AS email_domain,     -- PostgreSQL
    LEFT(name, 3)                               AS first_3,
    RIGHT(name, 3)                              AS last_3,
    REPLACE(phone, '-', '')                     AS clean_phone,
    TRIM(LEADING '0' FROM phone)                AS trimmed_phone,
    TRIM('  padded  ')                          AS trimmed,
    LPAD(CAST(id AS VARCHAR), 5, '0')           AS padded_id,        -- 00001
    POSITION('a' IN name)                       AS first_a,
    REVERSE(name)                               AS reversed
FROM employees;
```

DIALECTS MySQL SQL Server

- **Length:** MySQL `LENGTH` counts bytes (use `CHAR_LENGTH`); SQL Server uses `LEN`.
- **Concatenation:** in MySQL `||` means OR unless `PIPES_AS_CONCAT` is on, so use `CONCAT`. SQL Server concatenates with `+` or `CONCAT`.
- **Substrings and positions:** SQL Server uses `SUBSTRING(s, start, len)` and `CHARINDEX('@', email)`; MySQL uses `SUBSTRING_INDEX(email, '@', -1)` for the domain.
- **INITCAP:** not in MySQL or SQL Server.
- **CONCAT and NULLs:** `CONCAT` treats NULL as an empty string, but `||` returns NULL.

18.2 Pattern matching with regular expressions

POSTGRESQL 18.02

```postgresql
-- PostgreSQL: ~ (case-sensitive), ~* (case-insensitive)
SELECT
    name,
    CASE
        WHEN name ~ '^[A-Z][a-z]+ [A-Z][a-z]+$' THEN 'Valid Name'
        ELSE 'Invalid Format'
    END AS name_check
FROM customers;

SELECT email FROM employees WHERE email ~* '^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$';
SELECT REGEXP_REPLACE(phone, '[^0-9]', '', 'g') AS digits_only FROM employees;
```

MYSQL 18.03

```mysql
-- MySQL 8+
SELECT name FROM customers WHERE name REGEXP '^[A-Z][a-z]+ [A-Z][a-z]+$';
SELECT REGEXP_REPLACE(phone, '[^0-9]', '') AS digits_only FROM employees;
```

DIALECTS SQL Server

SQL Server (before 2025) has no regex; use `LIKE` with character classes such as `LIKE '[A-Z]%'`.

18.3 Date and time: current values and extraction

POSTGRESQL 18.04

```postgresql
SELECT
    CURRENT_DATE                    AS today,
    CURRENT_TIMESTAMP               AS now,
    order_date,
    EXTRACT(YEAR FROM order_date)   AS year,
    EXTRACT(MONTH FROM order_date)  AS month,
    EXTRACT(DAY FROM order_date)    AS day,
    EXTRACT(DOW FROM order_date)    AS day_of_week,   -- 0 = Sunday (PostgreSQL)
    EXTRACT(QUARTER FROM order_date) AS quarter,
    EXTRACT(WEEK FROM order_date)   AS week_number    -- ISO week
FROM orders;
```

18.4 Date arithmetic and truncation

POSTGRESQL 18.05

```postgresql
-- On orders
SELECT
    order_date,
    order_date + INTERVAL '7 days'          AS due_date,
    order_date - INTERVAL '1 year'          AS one_year_before,
    DATE_TRUNC('month', order_date)         AS month_start,
    DATE_TRUNC('week', order_date)          AS week_start,      -- Monday
    CURRENT_DATE - order_date               AS days_since_order -- date - date = integer days
FROM orders;

-- On employees
SELECT
    name,
    AGE(CURRENT_DATE, birth_date)                    AS age,            -- PostgreSQL interval
    EXTRACT(YEAR FROM AGE(CURRENT_DATE, birth_date)) AS age_years,
    CURRENT_DATE - hire_date                         AS days_employed,
    AGE(CURRENT_DATE, hire_date)                     AS tenure
FROM employees;

-- Format a date as text
SELECT TO_CHAR(order_date, 'YYYY-MM') AS year_month FROM orders;
```

MYSQL 18.06

```mysql
-- MySQL
SELECT
    DATE_ADD(order_date, INTERVAL 7 DAY)       AS due_date,
    DATE_SUB(order_date, INTERVAL 1 YEAR)      AS one_year_before,
    DATEDIFF(CURRENT_DATE, order_date)         AS days_since_order,
    DATE_FORMAT(order_date, '%Y-%m-01')        AS month_start,
    TIMESTAMPDIFF(YEAR, '1990-01-30', CURRENT_DATE) AS age_years
FROM orders;
```

SQL SERVER 18.07

```sql-server
-- SQL Server
SELECT
    DATEADD(day, 7, order_date)                          AS due_date,
    DATEDIFF(day, order_date, CAST(GETDATE() AS DATE))   AS days_since_order,
    DATETRUNC(month, order_date)                         AS month_start,   -- SQL Server 2022+
    FORMAT(order_date, 'yyyy-MM')                        AS year_month,
    DATEPART(year, order_date)                           AS year
FROM orders;
```

18.5 Date ranges and comparisons

POSTGRESQL 18.08

```sql
-- Last 30 days
SELECT * FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days';

-- Current month (index-friendly range form)
SELECT * FROM orders
WHERE order_date >= DATE_TRUNC('month', CURRENT_DATE)
  AND order_date <  DATE_TRUNC('month', CURRENT_DATE) + INTERVAL '1 month';

-- Year-over-year comparison by month
SELECT
    EXTRACT(MONTH FROM order_date) AS month,
    SUM(CASE WHEN EXTRACT(YEAR FROM order_date) = 2023 THEN total END) AS sales_2023,
    SUM(CASE WHEN EXTRACT(YEAR FROM order_date) = 2022 THEN total END) AS sales_2022
FROM orders
GROUP BY EXTRACT(MONTH FROM order_date)
ORDER BY month;
```

18.6 Numeric functions

POSTGRESQL 18.09

```sql
SELECT
    ROUND(1234.5678, 2)      AS rounded,        -- 1234.57
    ROUND(1234.5678)         AS to_integer,     -- 1235
    TRUNC(1234.5678, 1)      AS truncated,      -- 1234.5 (MySQL: TRUNCATE(x, 1))
    CEIL(4.1)                AS ceiling,        -- 5  (SQL Server: CEILING)
    FLOOR(4.9)               AS floor,          -- 4
    ABS(-7)                  AS absolute,       -- 7
    MOD(17, 5)               AS remainder,      -- 2  (also 17 % 5)
    POWER(2, 10)             AS power,          -- 1024
    SQRT(144)                AS square_root,    -- 12
    EXP(1)                   AS e,
    LN(10)                   AS natural_log,    -- SQL Server: LOG(10)
    LOG(10, 1000)            AS log_base_10,    -- 3  (argument order differs by database)
    SIGN(-3)                 AS sign,           -- -1
    RANDOM()                 AS random_0_to_1;  -- MySQL: RAND(), SQL Server: RAND()

-- Integer division trap: 7 / 2 = 3 in PostgreSQL and SQL Server
SELECT 7 / 2 AS int_div, 7 / 2.0 AS dec_div, CAST(7 AS DECIMAL) / 2 AS cast_div;
```

18.7 Type conversion (casting)

POSTGRESQL 18.10

```sql
SELECT
    CAST('2024-05-01' AS DATE)        AS as_date,
    CAST(salary AS INT)               AS salary_int,
    CAST(id AS VARCHAR(10))           AS id_text,
    '42'::INT                         AS pg_shorthand     -- PostgreSQL only
FROM employees;
```

DIALECTS MySQL SQL Server

SQL Server also has `CONVERT(INT, '42')` and `TRY_CAST` (returns NULL instead of an error). MySQL casts to `SIGNED` / `CHAR` rather than `INT` / `VARCHAR`: `CAST(x AS SIGNED)`, `CAST(x AS CHAR)`.

18.8 NULL handling summary

- Test with `IS NULL` / `IS NOT NULL`, never `= NULL`.
- `COALESCE(x, default)` replaces NULLs; `NULLIF(x, value)` creates them.
- Arithmetic with NULL gives NULL: `salary + NULL` is NULL.
- Aggregates skip NULLs; `COUNT(*)` counts rows, `COUNT(col)` counts non-NULL values; `AVG(col)` divides by the non-NULL count.
- `NOT IN (... NULL ...)` returns nothing; use `NOT EXISTS`.
- Sorting position of NULLs differs by database (see section 5).

POSTGRESQL 18.11

```sql
SELECT
    COUNT(*)                          AS all_rows,
    COUNT(bonus)                      AS rows_with_bonus,
    AVG(bonus)                        AS avg_of_non_null_bonuses,
    AVG(COALESCE(bonus, 0))           AS avg_treating_null_as_zero,
    SUM(salary + bonus)               AS sum_skips_rows_with_null_bonus,
    SUM(salary + COALESCE(bonus, 0)) AS correct_total_comp
FROM employees;
```

SECTION 19 · PART 05: FUNCTIONS & PATTERNS

19 Real-World Data Science Patterns

IN THIS SECTION Remove duplicates (keep the latest row per key) · Customer segmentation (spend quartile and recency) · Customer cohort analysis (monthly retention) · RFM analysis (Recency, Frequency, Monetary) · Time-series feature engineering · A/B test analysis · Running totals, share of total, gaps

19.1 Remove duplicates (keep the latest row per key)

POSTGRESQL 19.01

```postgresql
-- Find duplicates
SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1;

-- Keep one row per customer: their most recent order
SELECT *
FROM (
    SELECT o.*,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC, id DESC) AS rn
    FROM orders o
) t
WHERE rn = 1;
```

19.2 Customer segmentation (spend quartile and recency)

POSTGRESQL 19.02

```postgresql
WITH customer_stats AS (
    SELECT
        customer_id,
        COUNT(*)         AS purchase_count,
        SUM(total)       AS total_spent,
        MAX(order_date)  AS last_purchase,
        AVG(total)       AS avg_order_value
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    total_spent,
    NTILE(4) OVER (ORDER BY total_spent DESC) AS spending_segment,   -- 1 = top 25 %
    CASE
        WHEN CURRENT_DATE - last_purchase <= 30 THEN 'Active'
        WHEN CURRENT_DATE - last_purchase <= 90 THEN 'Lapsing'
        ELSE 'Inactive'
    END AS recency_segment
FROM customer_stats;
```

19.3 Customer cohort analysis (monthly retention)

POSTGRESQL 19.03

```sql
WITH cohort_assignments AS (
    SELECT
        user_id,
        CAST(DATE_TRUNC('month', MIN(signup_date)) AS DATE) AS cohort_month
    FROM users
    GROUP BY user_id
),
monthly_activity AS (
    SELECT
        c.cohort_month,
        CAST(DATE_TRUNC('month', a.activity_date) AS DATE) AS activity_month ,
        COUNT(DISTINCT a.user_id)                          AS active_users
    FROM cohort_assignments c
    JOIN user_activity a ON c.user_id = a.user_id
    GROUP BY c.cohort_month, CAST(DATE_TRUNC('month', a.activity_date ) AS DATE)
),
cohort_sizes AS (
    SELECT cohort_month, COUNT(*) AS cohort_size
    FROM cohort_assignments
    GROUP BY cohort_month
)
SELECT
    m.cohort_month,
    m.activity_month,
    -- months between the two dates, correct across year boundaries
    (EXTRACT(YEAR FROM m.activity_month) - EXTRACT(YEAR FROM m.cohort_month )) * 12
      + (EXTRACT(MONTH FROM m.activity_month) - EXTRACT(MONTH FROM m .cohort_month)) AS months_since_cohort,
    m.active_users,
    s.cohort_size,
    ROUND(m.active_users * 100.0 / s.cohort_size, 2) AS retention_rate
FROM monthly_activity m
JOIN cohort_sizes s ON s.cohort_month = m.cohort_month
ORDER BY m.cohort_month, m.activity_month;
```

19.4 RFM analysis (Recency, Frequency, Monetary)

Each customer gets a 1 to 5 score per dimension, where **5 is best** (most recent, most frequent, highest spend).

POSTGRESQL 19.04

```sql
WITH customer_rfm AS (
    SELECT
        customer_id,
        MAX(order_date) AS last_order_date,
        COUNT(*)        AS frequency,
        SUM(total)      AS monetary_value
    FROM orders
    WHERE order_date >= CURRENT_DATE - INTERVAL '3 years'
    GROUP BY customer_id
),
rfm_scores AS (
    SELECT
        customer_id,
        NTILE(5) OVER (ORDER BY last_order_date ASC) AS recency_score,   -- most recent = 5
        NTILE(5) OVER (ORDER BY frequency ASC)       AS frequency_score,
        NTILE(5) OVER (ORDER BY monetary_value ASC)  AS monetary_score
```

```
    FROM customer_rfm
),
rfm_cells AS (
    SELECT *,
           CONCAT(recency_score, frequency_score, monetary_score) AS rfm_cell
    FROM rfm_scores
)
SELECT
    rfm_cell,
    CASE
        WHEN rfm_cell IN ('555', '554', '545', '544') THEN 'Champions'
        WHEN rfm_cell LIKE '5__' THEN 'Loyal Customers'
        WHEN rfm_cell LIKE '__5' THEN 'Big Spenders'
        WHEN rfm_cell LIKE '1__' THEN 'Lost Customers'
        ELSE 'Needs Attention'
    END AS segment,
    COUNT(*) AS customers_count
FROM rfm_cells
GROUP BY rfm_cell
ORDER BY rfm_cell;
```

19.5 Time-series feature engineering

POSTGRESQL 19.05

```
WITH daily_metrics AS (
    SELECT
        CAST(DATE_TRUNC('day', event_time) AS DATE)              AS day,
        COUNT(*)                                                 AS event_count,
        COUNT(DISTINCT user_id)                                  AS daily_users,
        SUM(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS purchases
    FROM events
    GROUP BY CAST(DATE_TRUNC('day', event_time) AS DATE)
)
SELECT
    day,
    event_count,
    daily_users,
    purchases,
    -- Rolling averages
    AVG(event_count) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS weekly_avg_events,
    AVG(daily_users) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS weekly_avg_users,
    -- Growth rates
    event_count - LAG(event_count, 1) OVER (ORDER BY day) AS daily_event_change,
    (event_count - LAG(event_count, 1) OVER (ORDER BY day)) * 100.0
        / NULLIF(LAG(event_count, 1) OVER (ORDER BY day), 0) AS daily_event_growth_pct,
    -- Day-of-week patterns
    EXTRACT(DOW FROM day) AS day_of_week,
    AVG(event_count) OVER (PARTITION BY EXTRACT(DOW FROM day)) AS avg_dow_events,
    -- Cumulative metrics
    SUM(event_count) OVER (ORDER BY day) AS cumulative_events,
    SUM(daily_users) OVER (ORDER BY day) AS cumulative_user_days   -- not unique users
FROM daily_metrics
ORDER BY day;
```

19.6 A/B test analysis

Conversion rate is kept as a **proportion** (0 to 1) so the confidence-interval formula is valid; multiply by 100 only for display.

POSTGRESQL 19.06

```sql
WITH experiment_data AS (
    SELECT
        user_id,
        variant,
        -- Engagement metrics
        COUNT(DISTINCT session_id)                                  AS sessions,
        SUM(session_duration)                                       AS total_time,
        COUNT(CASE WHEN event_type = 'purchase' THEN 1 END)         AS purchases,
        COALESCE(SUM(CASE WHEN event_type = 'purchase' THEN revenue END), 0) AS total_revenue,
        -- Conversion flag
        MAX(CASE WHEN event_type = 'signup' THEN 1 ELSE 0 END)      AS converted
    FROM events
    WHERE experiment_name = 'homepage_design_2023'
      AND event_time >= '2023-10-01' AND event_time < '2023-11-01'
    GROUP BY user_id, variant
),
variant_stats AS (
    SELECT
        variant,
        COUNT(*)                        AS users,
        AVG(sessions)                   AS avg_sessions,
        AVG(total_time)                 AS avg_time,
        SUM(purchases)                  AS total_purchases,
        SUM(total_revenue)              AS total_revenue,
        SUM(converted)                  AS conversions,
        SUM(converted) * 1.0 / COUNT(*) AS conversion_rate        -- proportion 0..1
    FROM experiment_data
    GROUP BY variant
)
SELECT
    variant,
    users,
    ROUND(conversion_rate * 100, 2)                       AS conversion_pct,
    total_revenue,
    ROUND(total_revenue / users, 2)                       AS arpu,
    -- Lift versus the previous variant (A is the baseline when ordered by variant)
    ROUND((conversion_rate - LAG(conversion_rate) OVER (ORDER BY variant)) * 100, 2) AS lift_pct_points,
    -- 95 % confidence interval for the conversion rate (normal approximation)
    ROUND((conversion_rate - 1.96 * SQRT(conversion_rate * (1 - conversion_rate) / users)) * 100, 2) AS ci_lower_pct,
    ROUND((conversion_rate + 1.96 * SQRT(conversion_rate * (1 - conversion_rate) / users)) * 100, 2) AS ci_upper_pct
FROM variant_stats
ORDER BY variant;
```

19.7 Running totals, share of total, gaps

POSTGRESQL 19.07

```sql
-- Monthly revenue, running total and share of the year
SELECT
    DATE_TRUNC('month', order_date)                                     AS month,
    SUM(total)                                                          AS revenue,
    SUM(SUM(total)) OVER (ORDER BY DATE_TRUNC('month', order_date))     AS running_revenue,
    ROUND(SUM(total) * 100.0
          / SUM(SUM(total)) OVER (PARTITION BY EXTRACT(YEAR FROM DATE_TRUNC('month', order_date))), 2) AS pct_of_year
FROM orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;

-- Days with no sales row (gap detection with a generated calendar, PostgreSQL)
SELECT d::date AS missing_day
FROM generate_series(CURRENT_DATE - 89, CURRENT_DATE, INTERVAL '1 day') AS d
WHERE NOT EXISTS (SELECT 1 FROM daily_sales s WHERE s.sale_date = d::date);
```

06

Performance, Practice & Reference



20 **Performance Optimization** 57
Query optimization tips · Partitioning large tables · Common performance issues

21 **Best Practices** 58
Coding standards · Security considerations

22 **Database-Specific Extensions** 61
PostgreSQL: JSON, arrays, full-text search · MySQL specifics · SQL Server specifics

23 **Dialect Differences at a Glance** 63

24 **Quick Reference** 64
Order of execution · Statement families · Common data types · Key functions by category

25 **Key Tips for Data Professionals** 65

SECTION 20 · PART 06: PERFORMANCE, PRACTICE & REFERENCE

20 Performance Optimization

IN THIS SECTION Query optimization tips · Partitioning large tables · Common performance issues

20.1 Query optimization tips

POSTGRESQL 20.01

```postgresql
-- 1. Use EXPLAIN to see the plan; EXPLAIN ANALYZE also runs the query and shows real timings
EXPLAIN SELECT * FROM orders WHERE customer_id = 1;
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 1;

-- 2. Index frequently filtered / joined / sorted columns
CREATE INDEX idx_orders_customer ON orders(customer_id);

-- 3. Select only the columns you need instead of SELECT *
SELECT id, name, email FROM customers;

-- 4. Use LIMIT while developing queries
SELECT * FROM events LIMIT 100;

-- 5. Keep predicates "sargable": no functions on indexed columns
SELECT * FROM orders WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01';

-- 6. Pre-aggregate before joining big tables
WITH order_totals AS (
    SELECT customer_id, SUM(total) AS spent FROM orders GROUP BY customer_id
)
SELECT c.name, t.spent FROM customers c JOIN order_totals t ON t.customer_id = c.id;
```

> CAREFUL
>
> `EXPLAIN ANALYZE` really executes the statement. Wrap an `UPDATE` or `DELETE` in `BEGIN; ... ROLLBACK;` when analysing it. **Dialects:** MySQL 8.0.18+ also has `EXPLAIN ANALYZE`; SQL Server shows plans with `SET SHOWPLAN_XML ON` or "Include Actual Execution Plan" in SSMS.

20.2 Partitioning large tables

Partitioning splits one big table into pieces (by date range, list or hash) so queries touch only the relevant pieces.

POSTGRESQL 20.02

```postgresql
CREATE TABLE sales_partitioned (
    sale_id    INT,
    sale_date  DATE NOT NULL,
    amount     DECIMAL(12, 2)
) PARTITION BY RANGE (sale_date);

-- PostgreSQL needs the partitions created before rows can be inserted
CREATE TABLE sales_2023 PARTITION OF sales_partitioned
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
CREATE TABLE sales_2024 PARTITION OF sales_partitioned
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

INSERT INTO sales_partitioned VALUES (1, '2023-06-15', 100.00), (2, '2024-02-01', 250.00);
```

20.3 Common performance issues

1. **Missing indexes** on `WHERE`, `JOIN` and `ORDER BY` columns
2. **Unnecessary columns** in `SELECT`
3. **Complex subqueries** that could be CTEs, joins or temporary tables
4. **Functions on indexed columns** in `WHERE` clauses
5. **Implicit type conversions** in comparisons (for example comparing a text column to a number)
6. `SELECT DISTINCT` used to hide a join that fans out
7. **Large** `OFFSET` pagination instead of keyset pagination
8. **Stale statistics**: run `ANALYZE table_name` (PostgreSQL) or `ANALYZE TABLE` (MySQL)

SECTION 21 · PART 06: PERFORMANCE, PRACTICE & REFERENCE

21 Best Practices

IN THIS SECTION Coding standards · Security considerations

21.1 Coding standards

POSTGRESQL 21.01

```postgresql
-- Use meaningful aliases
SELECT
    cust.name          AS customer_name,
    ord.total          AS order_total,
    prod.product_name
FROM customers cust
JOIN orders ord        ON ord.customer_id = cust.id
JOIN order_items oi    ON oi.order_id = ord.id
```

```sql
JOIN products prod     ON prod.id = oi.product_id;

-- Format complex queries: one clause per line, CTEs for each step
WITH
    monthly_sales AS (
        SELECT
            DATE_TRUNC('month', order_date) AS month,
            SUM(total)                      AS total_sales
        FROM orders
        GROUP BY DATE_TRUNC('month', order_date)
    ),
    monthly_targets AS (
        SELECT month, 50000 AS target_amount        -- stand-in for a sales_targets table
        FROM monthly_sales
    )
SELECT
    ms.month,
    ms.total_sales,
    mt.target_amount,
    ms.total_sales - mt.target_amount AS variance
FROM monthly_sales ms
LEFT JOIN monthly_targets mt ON ms.month = mt.month
WHERE ms.month >= '2023-01-01'
ORDER BY ms.month DESC;
-- Comment complex logic
SELECT
    user_id,
    -- session length in minutes
    session_duration / 60.0 AS session_minutes,
    -- flag long sessions (> 10 minutes)
    CASE WHEN session_duration / 60.0 > 10 THEN 1 ELSE 0 END AS is_long_session
FROM events;
```

HABITS THAT PREVENT MISTAKES

- Write keywords in UPPER CASE and identifiers in snake_case.
- Always list columns in `INSERT`.
- Run the `WHERE` clause as a `SELECT` before any `UPDATE` or `DELETE`.
- Qualify columns with table aliases in joins.
- Use `IS NULL`, never `= NULL`.
- Store money as `DECIMAL`, never `FLOAT`.
- Use half-open date ranges (`>= start AND < end`).

21.2 Security considerations

POSTGRESQL 21.02

```
-- Use parameterized queries in application code (prevents SQL injection)
-- BAD:  "SELECT * FROM users WHERE email = '" + user_input + "'"
-- GOOD: SELECT * FROM users WHERE email = $1     (placeholder style varies by driver: $1, ?, %s, @email)
PREPARE find_user (TEXT) AS SELECT * FROM users WHERE email = $1;
EXECUTE find_user('user1@mail.com');

-- Limit data access with views
CREATE VIEW employee_directory AS
SELECT id, first_name, last_name, department, title
FROM employees;

-- Use read-only roles for analytics
CREATE ROLE analyst_role;
GRANT SELECT ON employee_directory TO analyst_role;
GRANT SELECT ON orders, customers TO analyst_role;
REVOKE SELECT ON customers FROM analyst_role;

-- Audit sensitive queries
CREATE TABLE query_audit (
    id           INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    username     VARCHAR(100) DEFAULT CURRENT_USER,
    query_text   TEXT,
    executed_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO query_audit (query_text) VALUES ('SELECT salary FROM employees');
```

SECTION 22 · PART 06: PERFORMANCE, PRACTICE & REFERENCE

22 Database-Specific Extensions

IN THIS SECTION PostgreSQL: JSON, arrays, full-text search · MySQL specifics · SQL Server specifics

22.1 PostgreSQL: JSON, arrays, full-text search

POSTGRESQL 22.01

```
-- JSON / JSONB operations
CREATE TEMP TABLE orders_json (id INT, data JSONB);
INSERT INTO orders_json VALUES
 (1, '{"name": "Acme", "address": {"city": "Nairobi"}, "items": [{"sku": "A1"}, {"sku": "B2"}]}');

SELECT
    data->>'name'                    AS name,          -- ->> returns text
    data->'address'->>'city'         AS city,          -- -> returns JSON
    jsonb_array_length(data->'items') AS item_count
FROM orders_json
WHERE data @> '{"name": "Acme"}';                      -- containment test (indexable with GIN)

-- Array functions
CREATE TEMP TABLE documents (id INT, title TEXT, tags TEXT[]);
INSERT INTO documents VALUES (1, 'Q1 report', ARRAY['finance', 'important']), (2, 'Memo', ARRAY['hr']);

SELECT
    title,
    tags,
    ARRAY_LENGTH(tags, 1)        AS tag_count,
    tags[1]                      AS primary_tag,       -- arrays are 1-based
    'important' = ANY(tags)      AS has_important_tag
FROM documents;

SELECT id, UNNEST(tags) AS tag FROM documents;         -- one row per array element

-- Full-text search
CREATE TEMP TABLE articles (title TEXT, content TEXT);
INSERT INTO articles VALUES ('Data science with SQL', 'Window functions and CTEs for analysis');

SELECT title, content
FROM articles
WHERE to_tsvector('english', title || ' ' || content)
      @@ to_tsquery('english', 'data & science');
```

22.2 MySQL specifics

MYSQL 22.02

```mysql
-- MySQL
-- Group concatenation
SELECT
    department,
    GROUP_CONCAT(DISTINCT last_name ORDER BY last_name) AS employees
FROM employees
GROUP BY department;

-- Window functions (MySQL 8.0+) with a named window
SELECT
    name,
    salary,
    ROW_NUMBER() OVER w AS row_num,
    AVG(salary)  OVER (PARTITION BY department) AS avg_dept_salary
FROM employees
WINDOW w AS (PARTITION BY department ORDER BY salary DESC);

-- JSON
SELECT JSON_EXTRACT(data, '$.address.city') AS city, data->>'$.name' AS name FROM orders_json;
```

22.3 SQL Server specifics

SQL SERVER 22.03

```sql
-- SQL Server
SELECT TOP 5 WITH TIES name, salary FROM employees ORDER BY salary DESC;
SELECT ISNULL(bonus, 0) AS bonus, IIF(salary > 50000, 'High', 'Low') AS band FROM employees;
SELECT JSON_VALUE(data, '$.address.city') AS city FROM orders_json;
SELECT * FROM orders ORDER BY id OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

SECTION 23 · PART 06: PERFORMANCE, PRACTICE & REFERENCE

23 Dialect Differences at a Glance

Task	POSTGRESQL	MYSQL 8	SQL SERVER
Limit rows	`LIMIT 10`	`LIMIT 10`	`TOP 10` / `OFFSET ... FETCH`
Auto-increment key	`GENERATED AS IDENTITY` / `SERIAL`	`AUTO_INCREMENT`	`IDENTITY(1,1)`
String concat	`CONCAT`, double-pipe operator	`CONCAT`	`CONCAT`, plus operator
String length	`LENGTH`	`CHAR_LENGTH`	`LEN`
NULL default	`COALESCE`	`COALESCE`, `IFNULL`	`COALESCE`, `ISNULL`
Current time	`NOW()`, `CURRENT_TIMESTAMP`	`NOW()`	`GETDATE()`, `SYSDATETIME()`
Add days	`d + INTERVAL '7 days'`	`DATE_ADD(d, INTERVAL 7 DAY)`	`DATEADD(day, 7, d)`
Days between	`d2 - d1`	`DATEDIFF(d2, d1)`	`DATEDIFF(day, d1, d2)`
Truncate date	`DATE_TRUNC('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATETRUNC(month, d)` (2022+)
Format date	`TO_CHAR(d, 'YYYY-MM')`	`DATE_FORMAT(d, '%Y-%m')`	`FORMAT(d, 'yyyy-MM')`
Case-insensitive LIKE	`ILIKE`	`LIKE` (collation)	`LIKE` (collation)
Regex	`~`, `REGEXP_REPLACE`	`REGEXP`, `REGEXP_REPLACE`	none (`LIKE '[A-Z]%'`)
Upsert	`ON CONFLICT`, `MERGE`	`ON DUPLICATE KEY UPDATE`	`MERGE`
Return changed rows	`RETURNING`	none	`OUTPUT`
String aggregation	`STRING_AGG`	`GROUP_CONCAT`	`STRING_AGG`
Median	`PERCENTILE_CONT` aggregate	none built in	`PERCENTILE_CONT ... OVER ()`
Full outer join	yes	no (emulate with `UNION`)	yes
Transactional DDL	yes	no	yes

SECTION 24 · PART 06: PERFORMANCE, PRACTICE & REFERENCE

24 Quick Reference

IN THIS SECTION Order of execution · Statement families · Common data types · Key functions by category

24.1 Order of execution

`FROM/JOIN` → `WHERE` → `GROUP BY` → `HAVING` → *window functions* → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT/OFFSET`

24.2 Statement families

DQL (query):	`SELECT`
DML (change data):	`INSERT`, `UPDATE`, `DELETE`, `MERGE`
DDL (change structure):	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`
DCL (permissions):	`GRANT`, `REVOKE`
TCL (transactions):	`BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`

24.3 Common data types

- `INT`, `BIGINT`: integers
- `DECIMAL(p, s)`: precise decimals
- `FLOAT`, `DOUBLE`: approximate numbers
- `VARCHAR(n)`: variable-length strings
- `TEXT`: long text
- `DATE`, `TIME`, `TIMESTAMP`: date/time
- `BOOLEAN`: true/false
- `JSON`, `JSONB`: JSON data
- `ARRAY`: arrays of values

24.4 Key functions by category

- **Aggregation:** `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `STDDEV`, `VARIANCE`, `PERCENTILE_CONT`, `STRING_AGG`
- **Window:** `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `PERCENT_RANK`
- **String:** `CONCAT`, `SUBSTRING`, `REPLACE`, `UPPER`, `LOWER`, `TRIM`, `LENGTH`, `POSITION`, `SPLIT_PART`
- **Date:** `EXTRACT`, `DATE_TRUNC`, `AGE`, `INTERVAL`, `CURRENT_DATE`, `CURRENT_TIMESTAMP`, `TO_CHAR`
- **Math:** `ROUND`, `CEIL`, `FLOOR`, `ABS`, `POWER`, `SQRT`, `LOG`, `LN`, `MOD`
- **Conditional:** `CASE`, `COALESCE`, `NULLIF`, `GREATEST`, `LEAST`
- **Conversion:** `CAST`, `::` (PostgreSQL), `CONVERT` / `TRY_CAST` (SQL Server)

SECTION 25 · PART 06: PERFORMANCE, PRACTICE & REFERENCE

25 Key Tips for Data Professionals

01 **Start with CTEs** for complex transformations; they're more readable and easier to debug step by step.

02 **Master window functions**; they're essential for feature engineering in-database.

03 **Always validate** data quality with `COUNT`s, `COUNT(DISTINCT)`, NULL counts and basic stats before analysing.

04 **Document assumptions** in comments within your SQL.

05 **Test with LIMIT** before running on full datasets.

06 **Learn your database's** specific functions and optimizations (see sections 22 and 23).

07 **Combine SQL with Python/R**: use each for what it does best.

08 **Master joins and their pitfalls** (fan-out, LEFT JOIN filters); they're fundamental to relational data.

09 **Practice performance tuning** with `EXPLAIN`; it's critical with large datasets.

10 **Practice date/time functions** for time-series analysis, and use half-open date ranges.

11 **Respect NULLs**: they change the results of comparisons, `NOT IN`, aggregates and joins.

12 **Keep learning**: SQL has deep capabilities for analytics.

These commands form the foundation for data extraction, transformation and analysis in SQL, which are critical skills for anyone working with relational databases.

A–C

Appendices

A **Sample Data (PostgreSQL)** 67

B **Corrections Made to the Original Notes** 69

C **Exporting This Cheat Sheet (from the original notes)** 70

APPENDIX A · APPENDICES

A Sample Data (PostgreSQL)

Run after the schema in section 1 to make every example return rows.

POSTGRESQL | LONG LINES WRAP A.01

```
INSERT INTO departments (id, name, location) VALUES
 (1,'Engineering','Nairobi'),(2,'Sales','Mombasa'),(3,'Marketing','Nairobi'),(4,'Finance','Kisumu'),(5,'HR',
|   NULL);

INSERT INTO employees (id, first_name, last_name, name, email, phone, gender, department, department_id,
|    manager_id, title, salary, bonus, experience_years, hire_date, birth_date) VALUES
 (1,'Amina','Otieno','Amina Otieno','amina@corp.com','0700-111-222','F','Engineering',1,NULL,'CTO',150000,
|   20000,15,'2015-03-01','1985-06-12'),
 (2,'Brian','Kamau','Brian Kamau','brian@corp.com','0700-222-333','M','Engineering',1,1,'Senior Engineer',
|   95000,NULL,8,'2018-07-15','1990-01-30'),
 (3,'Cynthia','Wanjiru','Cynthia Wanjiru','cynthia@corp.com',NULL,'F','Engineering',1,1,'Engineer',62000,5000,
|   3,'2022-01-10','1997-09-05'),
 (4,'David','Mwangi','David Mwangi','david@corp.com','0711-333-444','M','Sales',2,NULL,'Sales Director',110000,
|   15000,12,'2016-05-20','1983-11-22'),
 (5,'Esther','Achieng','Esther Achieng','esther@corp.com','0711-444-555','F','Sales',2,4,'Account Manager',
|   58000,NULL,4,'2021-02-01','1995-04-17'),
 (6,'Felix','Njoroge','Felix Njoroge',NULL,'0722-555-666','M','Sales',2,4,'Sales Rep',28000,2000,1,
|   '2024-03-11','2000-12-01'),
 (7,'Grace','Muthoni','Grace Muthoni','grace@corp.com','0722-666-777','F','Marketing',3,NULL,'Marketing Lead',
|   85000,8000,9,'2017-09-04','1988-02-14'),
 (8,'Hassan','Abdi','Hassan Abdi','hassan@corp.com',NULL,NULL,'Marketing',3,7,'Analyst',45000,NULL,2,
|   '2023-06-19','1998-08-08'),
 (9,'Irene','Chebet','Irene Chebet','irene@corp.com','0733-777-888','F','Finance',4,NULL,'CFO',140000,18000,14,
|   '2014-11-30','1982-03-03'),
 (10,'James','Kiprop','James Kiprop','james@corp.com','0733-888-999','M','Finance',4,9,'Accountant',52000,0,5,
|   '2020-08-24','1993-10-10'),
 (11,'Khadija','Omar','Khadija Omar','khadija@corp.com','0744-999-000','F','HR',5,NULL,'HR Manager',70000,NULL,
|   7,'2019-04-01','1991-07-21'),
 (12,'Leo','Ndegwa','Leo Ndegwa','leo@corp.com',NULL,'M','Engineering',1,2,'Intern',0,NULL,0,CURRENT_DATE - 20,
|   '2003-05-05');

INSERT INTO customers (id, name, email, phone, city, status, signup_date) VALUES
 (1,'Acme Ltd','ops@acme.co.ke','020-100','Nairobi','active','2022-01-15'),
 (2,'Baraka Stores','info@baraka.co.ke',NULL,'Mombasa','active','2022-06-01'),
 (3,'Cedar Farms',NULL,'020-300','Nakuru','inactive','2021-11-20'),
 (4,'Duka Online','hello@duka.ke','020-400','Nairobi','active','2023-03-08'),
 (5,'Elimu Books','sales@elimu.ke',NULL,'Kisumu','active','2023-09-12'),
 (6,'Fahari Hotels',NULL,NULL,'Mombasa','inactive','2024-01-05');

INSERT INTO categories (id, category_name) VALUES (1,'Electronics'),(2,'Books'),(3,'Groceries'),(4,
|   'Clearance');

INSERT INTO products (id, product_name, category_id, price, units_sold, profit_margin) VALUES
 (1,'Laptop',1,85000,120,0.25),(2,'Phone',1,30000,2400,0.35),(3,'Headphones',1,4500,800,0.40),
 (4,'SQL for Analysts',2,2500,1500,0.50),(5,'Novel',2,1200,300,0.20),(6,'Coffee 1kg',3,1800,5000,0.15),
 (7,'Old Charger',4,500,60,0.05),(8,'Mystery Box',NULL,999,10,0.10);

INSERT INTO orders (id, customer_id, order_date, total, status) VALUES
 (1,1,'2022-02-01',85000,'shipped'),(2,1,'2023-01-15',30000,'shipped'),(3,2,'2023-02-20',4500,'shipped'),
 (4,2,'2023-07-04',12000,'shipped'),(5,3,'2022-12-10',1800,'cancelled'),(6,4,'2023-03-09',115000,'shipped'),
 (7,4,'2023-11-23',2500,'shipped'),(8,5,'2024-01-10',3700,'shipped'),(9,1,'2024-02-14',9000,'shipped'),
 (10,2,'2024-05-30',30000,'paid'),(11,4,CURRENT_DATE - 40,1800,'paid'),(12,1,CURRENT_DATE - 25,85000,'paid'),
 (13,2,CURRENT_DATE - 10,4500,'paid'),(14,4,CURRENT_DATE - 5,6100,'paid'),(15,5,CURRENT_DATE - 2,2500,'paid'),
 (16,1,CURRENT_DATE - 1,1200,'paid');

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
```

```
  (1,1,1,85000),(2,2,1,30000),(3,3,1,4500),(4,4,4,2500),(4,6,1,1800),(5,6,1,1800),(6,1,1,85000),(6,2,1,30000),
  (7,4,1,2500),(8,4,1,2500),(8,5,1,1200),(9,3,2,4500),(10,2,1,30000),(11,6,1,1800),(12,1,1,85000),
  (13,3,1,4500),(14,4,1,2500),(14,6,2,1800),(15,4,1,2500),(16,5,1,1200);
INSERT INTO daily_sales (sale_date, revenue)
SELECT d::date, ROUND((1000 + 400 * sin(n / 3.0) + n * 15)::numeric, 2)
FROM generate_series(CURRENT_DATE - 89, CURRENT_DATE, INTERVAL '1 day') WITH ORDINALITY AS g(d, n);

INSERT INTO users (user_id, email, signup_date)
SELECT n, 'user' || n || '@mail.com', DATE '2024-01-01' + ((n % 4) * 31)
FROM generate_series(1, 40) AS n;

INSERT INTO user_activity (user_id, activity_date)
SELECT u.user_id, u.signup_date + (m * 30)
FROM users u CROSS JOIN generate_series(0, 5) AS m
WHERE (u.user_id + m) % 3 <> 0;

INSERT INTO events (user_id, event_type, event_time, session_id, session_duration, revenue, experiment_name,
|    variant)
SELECT (n % 40) + 1,
       CASE WHEN n % 7 = 0 THEN 'purchase' WHEN n % 5 = 0 THEN 'signup' ELSE 'view' END,
       TIMESTAMP '2023-10-01 08:00' + (n * INTERVAL '7 hours'),
       n / 3,
       60 + (n * 37) % 900,
       CASE WHEN n % 7 = 0 THEN 500 + (n % 11) * 100 END,
       'homepage_design_2023',
       CASE WHEN n % 2 = 0 THEN 'A' ELSE 'B' END
FROM generate_series(1, 400) AS n;

-- Move identity counters past the explicit ids used above
SELECT setval(pg_get_serial_sequence('departments', 'id'), 100);
SELECT setval(pg_get_serial_sequence('employees', 'id'), 100);
SELECT setval(pg_get_serial_sequence('customers', 'id'), 100);
SELECT setval(pg_get_serial_sequence('categories', 'id'), 100);
SELECT setval(pg_get_serial_sequence('products', 'id'), 100);
SELECT setval(pg_get_serial_sequence('orders', 'id'), 100);
```

APPENDIX B · APPENDICES

B Corrections Made to the Original Notes

01 The derived-table example selected `dept_name`, a column the subquery never produced; it now selects `department`.

02 "Date Arithmetic" and "Date operations" mixed columns from different tables (`order_date` and `hire_date` from `employees`); they are now split into an orders query and an employees query.

03 `DATE_PART('day', CURRENT_DATE - hire_date)` fails in PostgreSQL because subtracting two dates already returns an integer; it is now `CURRENT_DATE - hire_date` (same fix in customer segmentation).

04 Cohort analysis computed months since the cohort as a month-number difference, which breaks across years (January minus December = -11). It now uses `(year diff) * 12 + (month diff)`.

05 RFM analysis ranked recency with `ORDER BY last_order_date DESC`, which gave the most recent customers a score of 1 while the segment rules treated 5 as best. Recency now sorts ascending so 5 means most recent, and the scores are reused instead of recomputing `NTILE` three more times.

06 The A/B test stored the conversion rate as a percentage (times 100) and then used it in a confidence-interval formula that needs a proportion. It now keeps a 0 to 1 proportion and converts to percent only for display.

07 Percent-change calculations now guard against division by zero with `NULLIF(..., 0)`.

08 `ROW_NUMBER() ... AS rank` renamed (`rank` is a reserved word in MySQL 8 and the alias was misleading).

09 "Last 7 days" with `ROWS BETWEEN 6 PRECEDING` is only 7 days if there is exactly one row per day; a `RANGE ... INTERVAL '6 days'` version was added.

10 `SUM(daily_users) OVER (...)` was labelled "cumulative_users", but summing daily distinct users double counts; it is now labelled `cumulative_user_days`.

11 `WHERE condition1 AND condition2 OR condition3` was shown without explaining that `AND` binds first; parentheses are now explained.

12 The partitioning example now creates the partitions; in PostgreSQL inserts fail without them.

13 Placeholders such as `table_name`, `condition` and `value1` were replaced with real tables from the sample schema so that every example runs.

14 Text-conversion artefacts in the original (stray `\*` escapes, missing spaces in `SELECT* FROM`, and auto-links such as `[a.id](http://a.id)` inside code) were removed.

15 Dialect-specific syntax (`AGE`, `DATE_TRUNC`, `INTERVAL '30 days'`, `~` regex, `ALTER COLUMN ... TYPE`, `STDDEV`, `PERCENTILE_CONT`) is now labelled, with MySQL and SQL Server equivalents.

APPENDIX C · APPENDICES

C Exporting This Cheat Sheet (from the original notes)

To convert this to PDF:

1. **Copy the entire content** above.
2. **Use one of these methods**:
   - **Method 1, Markdown to PDF (recommended):** save it as a `.md` file and use **Pandoc**: `pandoc sql-cheatsheet.md -o sql-data-science.pdf`. Or use an online converter such as **Markdown to PDF**.

- **Method 2, Word/Google Docs:** paste into Word or Google Docs, apply heading formatting, use a fixed-width font for code blocks, and export as PDF.
- **Method 3, direct printing:** copy into a text editor and print to a PDF printer.
- **Method 4, Jupyter Notebook:** display the Markdown in a notebook, then export the notebook as PDF.

PYTHON C.01

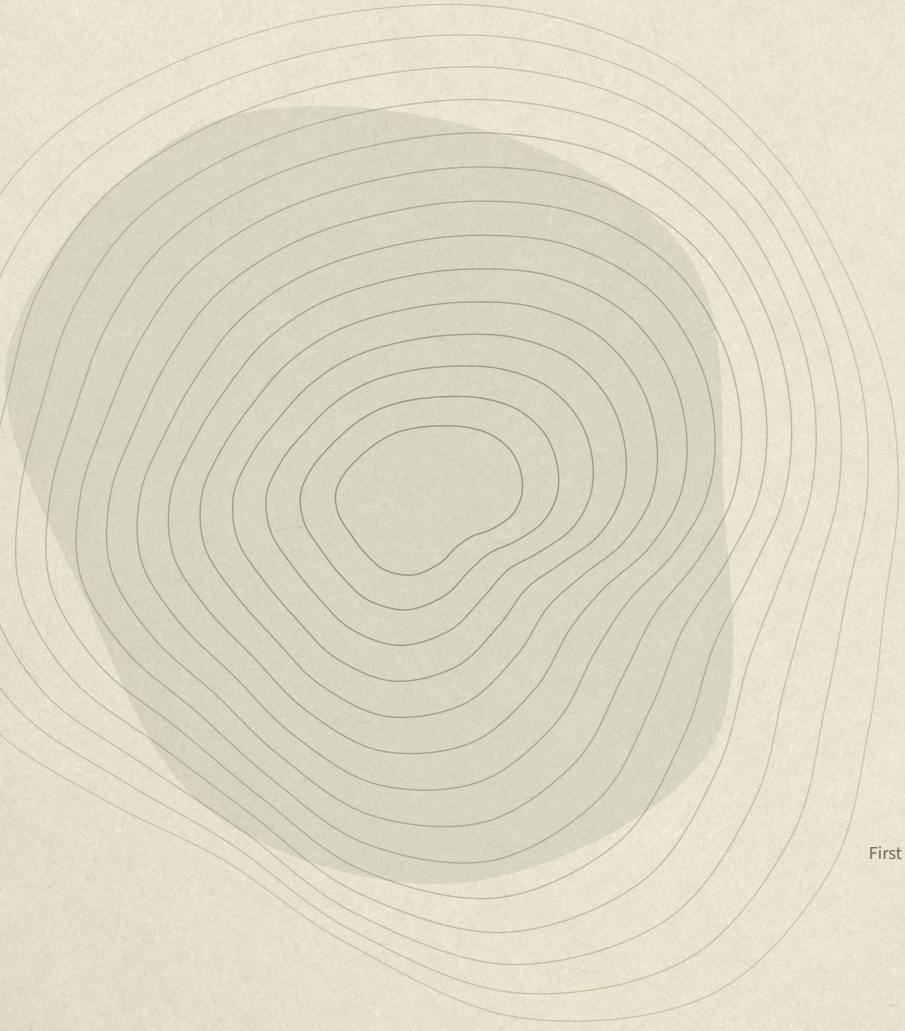
```python
# Create a Jupyter notebook and export as PDF
import pandas as pd
from IPython.display import Markdown

# Display as markdown then export
```

SQL Cheat Sheet

The Complete Reference for Data Work

FROM → WHERE → GROUP BY → HAVING → WINDOW → SELECT → DISTINCT → ORDER BY → LIMIT



WRITTEN AND PUBLISHED BY

Sharahbill Mohammed Abdi

First edition, 2026 · © 2026 Sharahbill Mohammed Abdi. All rights reserved.